Outage-Aware Reporting and Loss Recovery for Energy-Efficient River Water-Quality Telemetry over LPWANs

Kofi Sarpong Adu-Manu and Stephen Kwaku Azongo

***Abstract*—Battery-powered river water-quality nodes lose data, yet retransmission, buffering and reconstruction are analysed for short, independent losses. This paper measures loss in four real systems, two water-quality deployments and two LoRaWAN networks. Losses separate into short radio losses and rarer long delivery gaps, which hold most of the missing data in the water-quality deployments; a geometric gap-length model fits worst on every record. Replayed through a measurement-parameterised LoRaWAN and NB-IoT energy model, retransmission recovers under a fifth of the missing readings even under optimistic retry timing, at 2.3–7.3 the energy of unconfirmed reporting on LoRaWAN, whereas buffering with back-off recovers every generated reading at an energy close to that of recovering nothing. Neither simple nor deep reconstruction keeps most day-long gaps within tolerance. Bounded event-driven batching combines hourly batches, immediate alarms, a bounded-error trigger and buffering with back-off. Tested with frozen rules on held-out data from 33 river sites, it matches per-reading reporting on alarm latency and real-time picture, while per-reading retransmission uses 1.34×–2.55× as much energy. Recovery of many nodes after a shared outage needs randomised probes, and large SF12 cells are limited by the gateway's acknowledgements.**

***Index Terms*—Delivery gaps, network outages, store-and-forward, ARQ, LoRaWAN, NB-IoT, energy efficiency, water-quality monitoring, data reconstruction, event detection.**

I. Introduction

RIVER and reservoir monitoring with in-situ sensor nodes has to work where field visits are expensive and power is scarce [1], [2]. A typical node samples temperature (T), specific conductance (EC), dissolved oxygen (DO) and pH every 5–30 minutes and sends the readings over a cellular or low-power wide-area network (LPWAN). Such nodes lose data. The InWaterSense deployment on the Sitnica river in Kosovo delivered 19,090 of its expected readings and missed 16,230 (46.0%) [3]; the CSIRO water-quality network in Australia averaged about 64% delivery because the gateway's connection to the server "was down occasionally" [4]; and deployment reports describe gateways that lose power every night [5], sinks that stop reconnecting until someone reboots them [6], [7], outages lasting days to weeks [8], and prolonged outages of cellular backhaul in remote areas [9].

There are three standard responses to a lost reading. A node can *retransmit* it until it is acknowledged (automatic repeat request, ARQ); it can *buffer* it and *backfill* it later; or the server can *reconstruct* it from the node's own history or from neighbouring sites. Which response is right depends on how data are actually lost. Retransmission helps against a short radio fade but spends energy in vain against a gateway that is down for a day; reconstruction can fill a ten-minute gap but not a week. Yet the analyses behind these remedies assume losses that are short: estimation over lossy links is analysed with independent or short-memory (Markov fading) packet errors [10]–[12], and state-of-the-art imputation methods are evaluated on random masks or on simulated failures of a few hours [13]–[16]. Kong et al. [13] describe long losses caused by damaged or depleted nodes, but as named patterns, not as measured durations. How long real delivery gaps last, how much of the missing data they hold, and what that implies for the energy and timeliness of each remedy have not been measured for environmental telemetry.

The contribution of this paper is not a new retransmission or prediction primitive: ARQ, buffering, back-off, batching and error-bounded (dual-prediction) reporting are all established. It is an evidence-to-design chain for outage-aware, energy-efficient telemetry. Real delivery records are used to measure how long losses last and where the missing data are. The records are then replayed through a measurement-parameterised LPWAN energy model, with explicit bounds on what the records cannot tell (why a reading is missing, and when within a slot the network returned), to decide when retransmission, buffering, batching and reconstruction pay. Finally, the reporting design that follows from these findings is fixed on one year of river data and tested on a held-out year. Four research questions structure the paper:

- RQ1 In the records studied, how long are the gaps in delivery, how much of the missing data do long gaps hold, and is a geometric (memoryless) run-length model adequate?
- RQ2 Under these gaps, what do retransmission, buffering with back-off and batching each recover, and at what energy and delay on LoRaWAN and NB-IoT?
- RQ3 Can reconstruction, from a site's own history or from neighbouring sites on the same river, replace delivery during a gap?
- RQ4 Can one reporting scheme raise alarms as fast as per-reading reporting and keep the server's real-time picture within a stated tolerance as often, while using substantially less energy than per-reading retransmission, on data not used to design it?

The contributions follow the chain.

- *Empirical loss structure* (Sec. IV). In four records, two

K. S. Adu-Manu and S. K. Azongo are with the Department of Computer Science, University of Ghana, Legon, Ghana. Corresponding author: K. S. Adu-Manu.

water-quality deployments and two LoRaWAN networks, losses separate into short radio losses and rare long delivery gaps or system outages, and in the water-quality records gaps of 6 h or more hold most of the missing readings. A geometric run-length model is strongly unsupported relative to the alternatives tested, and fails worst in the long-gap tail that decides between remedies. Where the records allow it, the cause of the long gaps is examined: node-wide silence on the single-node record, and mostly a shared gateway or backhaul path on the two-node record.

- *Consequence analysis with explicit bounds* (Secs. V and VI). Replaying the records shows that ARQ recovers under a fifth of the missing readings even under optimistic assumptions on retry timing, at 2.3–7.3 the energy of unconfirmed reporting on LoRaWAN, whereas buffering with probe back-off recovers every reading the node generated at an energy close to that of recovering nothing in the model, and within 0.50×–2.08× of it under wide parameter uncertainty. The results are bracketed for the unknown cause of gaps, the timing of retries within a slot, finite storage and the duty cycle, the back-off cap, and many nodes recovering at once. For reconstruction, the evaluated estimators, from a site's own day–night (diel) pattern to two contemporary deep imputers, keep few day-long gaps within tolerance, and a test against the last value alone overstates the value of neighbouring sites.
- *An outage-aware reporting design, confirmed on held-out data* (Sec. VII). Bounded event-driven batching combines hourly batches, immediate uploads on a node-side alarm, a bounded-error trigger that mirrors the server's estimate, and buffering with back-off. It was developed on 30 USGS river sites in 2023 and then evaluated, with its parameters and decision rules frozen, on 33 sites in 2024.

Scope. The loss evidence comes from four records, and the two water-quality records cannot fully separate a network outage from a node that stopped producing readings; the paper therefore speaks of delivery gaps, and treats buffering's recovery as an upper bound for gaps of unknown cause. Energy comes from a model parameterised by published measurements, not from hardware built for this study; the claims are ratios between strategies on the same radio, tested against wide parameter uncertainty. The design is evaluated per node, with a separate multi-node model of recovery after a shared outage.

The rest of the paper is organised as follows. Sec. II reviews related work, and Sec. III describes the data, models and assumptions. Secs. IV–VII answer RQ1–RQ4 in turn, and Sec. VIII discusses practical guidance, validity and limitations.

II. Related Work

A. How sensor networks lose data

Early measurements of dense wireless sensor networks showed that packet delivery varies widely between links and over time [17]. Kong et al. [13] analysed three real sensor datasets with losses of 23%, 35% and 64%, and synthesised five named loss patterns, from random element loss to "successive loss in a row" when a node is damaged or out of energy; the patterns are motivated by their causes, but their durations are not measured. Deployment reports describe the long-gap regime anecdotally. In the CSIRO water-quality network, losses occurred at the gateway when its server connection was down, and end-to-end recovery from RAM copies improved delivery by at most 10.6% [4]. A solar-powered gateway lost power every night until after sunrise [5]; a GPRS sink failed to reconnect after disconnections until it was rebooted remotely [6]; a gateway's modem silently disappeared from its operating system until a daily watchdog reset was added [7]; and a sub-Arctic LoRaWAN deployment saw outages of days to weeks from gateway, power and connectivity faults [8]. For LoRaWAN, public datasets now allow loss to be measured from frame counters: LoED records the traffic of nine gateways in London [18], and a 16-month deployment at the University of Virginia (UVA) records ten sensors, three gateways and co-located weather [19]. No previous study appears to have measured gap durations across several such records, fitted models to them, and asked what they imply for the choice of loss-handling remedy.

B. Estimation and retransmission under loss

Kalman filtering with intermittent observations [10] established that estimation error stays bounded only if observations arrive often enough, under independent (Bernoulli) arrivals. Work on real-time remote estimation asks whether a failed packet should be retransmitted or replaced by a fresh measurement, with independent or finite-state Markov fading channels [11], [12], and finds switching-type optimal policies. Age of information formalises timeliness [20], and semantic-aware remote estimation schedules redundant sensors that observe the same source [21]. These works supply the vocabulary of the question studied here, retransmit or refresh, but not its setting: multi-day gaps, battery-limited LPWAN radios, and environmental signals with strong daily cycles. On LoRaWAN, application-layer coding recovers frames lost to the radio from redundancy in later frames [22]; it addresses radio loss, not gaps in which nothing gets through.

C. Reconstruction of missing sensor data

Compressive and low-rank data gathering exploit spatial and temporal correlation to recover readings that were not collected or were lost [13], [23]–[25]. Deep spatio-temporal imputation is the current state of the art: graph neural networks such as GRIN [14], SPIN [15] and StemGNN [26], self-attention models such as SAITS [27] and ImputeFormer [28], and causality-aware graph models that discount spurious spatial correlation [29]. GRIN and SPIN were evaluated mainly on random point losses and on simulated block failures of 12–48 steps (1–4 h at 5-minute steps), and deep compressed sensing for urban environmental monitoring on completely random losses [16]. The simulated failures thus last hours, not days, and the GRIN and SPIN evaluations include no per-site diel (seasonal-naive) baseline, although several of their

signals have strong day–night cycles. River networks have a specific spatial structure, along the flow, that geostatistical models represent explicitly [30]. This paper tests upstream and downstream neighbours on the same river, and two of the deep imputers, against a site's own diel pattern, on gaps of up to three days.

D. Store-and-forward in practice, and LPWAN energy

Buffering on the node and forwarding later is established practice. The CSIRO team proposed flash buffering "to handle network outage better" [4]; the potato-field deployment planned node-side logging with a daily network-wide acknowledgement [5]; SeNDT stored all lake water-quality data on the node for later collection from the shore [31]; Arctic tundra nodes keep append-only logs and advance their upload position only after an acknowledgement [32]; sub-Arctic nodes oversample and buffer in flash [8]; and a disruption-tolerant gateway stores LoRaWAN messages while its cellular backhaul is down [9]. The same reports document the pitfalls: draining a backlog can starve live traffic [6], and the buffer's storage can itself fail [7], [32]. What is missing is a quantitative comparison of these remedies, on energy, completeness and timeliness, under measured gap processes. The energy model of this paper rests on published measurements: state-by-state LoRaWAN current profiles [33], NB-IoT energy per report with release assistance [34], and NB-IoT power states [35]. Analytical models describe the NB-IoT latency–energy trade-off [36], and side-by-side measurements compare LoRaWAN and NB-IoT energy in a developing-country setting [37].

E. Prediction-based reporting

Dual prediction schemes run the same predictor at the node and the server and transmit only when the server's prediction would leave a tolerance [38], [39]; recent work combines dual prediction with adaptive sampling under a hard energy budget [40]. These schemes are usually evaluated with reliable or independently lossy links, and their purpose is to suppress reports. This paper uses the same mechanism for a different purpose: to decide when an hourly batch must leave early because the server's estimate would otherwise leave tolerance, and it makes the mechanism model when data arrive, not only whether.

Table I places the closest work against the questions of this paper.

III. Data and Models

Terms. A *reading* is one sampling cycle of a node: the four parameters taken at one instant, which travel together. Time is divided into *slots* at the reporting cadence, one reading per slot. A *delivery gap* is a run of consecutive slots for which no reading reached the server, whatever the cause. A *system outage* is a delivery gap verified to affect every node of a network (its gateway, backhaul or server); a *gateway outage* affects the nodes of one gateway; a *radio loss* is a frame lost while the rest of the path worked. These terms are used with these meanings throughout.

A. Delivery records

The study uses four real records of what a network delivered (Table II).

InWaterSense (Kosovo). Node 1 of the InWaterSense deployment on the Sitnica river [3], [41] reported about every 611 s from May 2015 to January 2016. The node sends each parameter as a separate message, seconds apart; the messages of one sampling instant are grouped into a reading, and a slot counts as delivered if a reading arrived for it. The node delivered 19,090 readings and missed 16,230 slots (46.0%).

Weija (Ghana). The Weija deployment [2] logged 9 parameter streams from one node from October 2018 to March 2019. Only records not marked as imputed count as delivered. Gap statistics and models use the conductivity stream (32.5-minute cadence), and the other streams are used to check that its pattern is typical and to examine the cause of its gaps (Sec. III-B). Even among the records not marked as imputed, 10.4% of the conductivity values carry more than the three decimals the logger otherwise records (up to 14), a sign of unflagged interpolation. Such values hide some short gaps, so counts of short gaps are lower bounds; long gaps are not affected.

UVA (Virginia, USA). Ten LoRaWAN sensors reporting every 5 min through three gateways on The Things Network, from February 2024 to June 2025 [19], [42]. Missing frames are found from frame-counter gaps. A missing frame is classified as a *system* loss if no sensor at all reached the database in its 10-minute bin (network server, internet or the operators' logging pipeline), as a *gateway* loss if all of the sensor's serving gateways were silent, and as a *radio* loss otherwise.

LoED (London, UK). The LoRaWAN at the Edge Dataset [18], [43] logs 5,118,817 uplink frames from 8,503 device addresses heard by 9 gateways. The devices belong to other operators, so a frame counts as lost when no LoED gateway heard it, and most devices are simply far away. Only *well-heard* devices are therefore used, those with at least 200 frames and a radio loss below 20% (179 of 1,780), as a proxy for a node's own link, and a missing frame is classified as a gateway loss if none of the device's serving gateways logged anything in that hour.

Frame counters. Both LoRaWAN analyses count a frame once per device and frame counter, whichever gateways heard it and however often it was retransmitted. A counter decrease or a jump beyond a threshold (5,000 frames on UVA, 1,000 on LoED) is treated as a reset or rejoin, not as loss. On UVA, 1 counter decrease occurred, no (sensor, counter) pair was received at two different times (no counter reuse), every sensor kept its 5-minute cadence in every month, and a threshold of 1,000 instead of 5,000 changes the count of missing frames by 2%. On LoED, 81 of the 179 well-heard devices show a reset, and their pooled loss is 10.0% with a threshold of 100 and 10.9% with 1,000.

B. What the water-quality records say about the cause of a gap

Neither water-quality record has a node status log, so a gap in which the network was down cannot be told apart directly

TABLE I
CLOSEST RELATED WORK. MEASURED: QUANTIFIED FROM DATA IN THE WORK; REPORTED: DESCRIBED, NOT QUANTIFIED; –: NOT ADDRESSED.

Work	Gap durations	Retransmission	Buffering / backfill	Energy	Long-gap reconstruction	Event alarms	Held-out test
Kong et al. [13]	reported (patterns)	–	–	–	low-rank, random/row losses	–	–
Le Dinh et al. [4]	reported	–	RAM copies, measured	–	–	–	–
SensorScope [6], Arctic [32], sub-Arctic [8]	reported	–	deployed	–	–	–	–
Huang et al. [11], [12]	– (i.i.d./Markov)	optimal policy	–	–	–	–	–
DaRe [22]	–	coding, radio loss	–	–	–	–	–
GRIN [14], SPIN [15]	–	–	–	–	simulated blocks of 1–4 h	–	–
Arnaiz et al. [40]	–	–	–	modelled	–	–	–
This paper	measured, 4 records	bracketed replay	with storage, duty cycle	modelled, ratios	up to 72 h, incl. deep	yes	2024 sites

TABLE II
DELIVERY RECORDS. LOSS: SHARE OF EXPECTED READINGS (INWATERSENSE, WEIJA) OR FRAMES (UVA, LOED) THAT DID NOT ARRIVE. GAPS: RUNS OF CONSECUTIVE MISSING SLOTS OR FRAMES. LOST IN $\geq$1 D: SHARE OF THE MISSING READINGS THAT FALL IN GAPS OF ONE DAY OR MORE, $S(1\,\mathrm{d})$ IN (1). WEIJA: CONDUCTIVITY STREAM (71.0%–100.0% OVER ALL 9 STREAMS). LOED: WELL-HEARD DEVICES ONLY (RADIO LOSS). GW: GATEWAYS.

Record	Type	Slot	Loss	Gaps	Lost in $\geq$1 d
InWaterSense	water quality	611 s	46.0%	1,619	50.7%
Weija	water quality	32.5 min	15.4%	42	82.9%
UVA	LoRaWAN, 3 GW	5 min	4.5%	36,519	23.6%
LoED	LoRaWAN, 9 GW	varies	9.6%	56,276	–

from one in which the node produced nothing. Two partial checks are possible.

Weija: the node's other streams. The node's sensors share one logger and one uplink. A conductivity slot in which another stream delivered a genuine reading (not imputed, at most three decimals, not on an interpolated line) had a live node and a working uplink. Of the 916 missing conductivity slots, 84% were node-silent, with no genuine reading from any stream, and 16% were conductivity-only gaps with the link up. In gaps of 6 h or more only 6% of the slots had another stream delivering: Weija's long gaps are silences of the whole node, which may be the node, its logger or its uplink. The node-level record (a slot is delivered if any stream delivered) loses 13.9% of the slots in 31 gaps, with 81% of the missing slots in gaps of a day or more, close to the conductivity record's 82.9%.

InWaterSense: the second node. Both InWaterSense nodes reached the server through one gateway with one GPRS link, housed in node 2's manhole [3]. While node 2 was operating (May–August 2015, a window that holds 57% of node 1's missing slots), a node-1 gap during which node 2 kept delivering cannot be a gateway, GPRS or server outage. Node 2 was silent during 79% of node 1's missing slots, far more often than when node 1 delivered. Correcting for node 2's own, local rate of silence, gaps of 6 h or more were about 74% shared-cause (gateway, backhaul, server or a shared power failure) and 26% specific to node 1, while gaps shorter than 6 h were 63% node-specific, as radio losses would be.

These checks support reading most long gaps as failures of the path to the server, but they cannot rule out that some were nodes that stopped producing readings. Sec. V therefore treats buffering's recovery as an upper bound and repeats the replay with up to half of the long-gap slots reassigned to a node that was off.

C. Gap-length models

A delivery record is reduced to alternating *up-runs* (consecutive delivered slots) and gaps (consecutive missing slots), in units of the record's slot. Let $k_1, \ldots, k_n$ be the gap lengths and $h(k)$ the duration of a gap of k slots. Because the quantity that matters for loss handling is where the missing data are, not where the gap events are, the main statistic is the share of missing slots that fall in gaps of at least a duration L,

$$S(L) = \frac{\sum_{i:\, h(k_i) \geq L} k_i}{\sum_{i=1}^{n} k_i}, \tag{1}$$

reported for $L = 6$ h and 1 day with bootstrap 95% confidence intervals [44] (gaps resampled with replacement, 5,000 draws). A gap is bounded by delivered slots on both sides, so every gap length is fully observed; only the first and last up-runs are censored, and up-runs are not modelled.

Four models are fitted to the gap lengths $k \geq 1$ by maximum likelihood:

$$P_{\mathrm{geo}}(k) = (1-q)\, q^{k-1}, \tag{2}$$

$$P_{\mathrm{mix}}(k) = w\,(1-q_1)\, q_1^{k-1} + (1-w)\,(1-q_2)\, q_2^{k-1}, \tag{3}$$

$$P_{\ln}(k) = \frac{\Phi\!\left(\frac{\ln(k+\frac{1}{2})-\mu}{\sigma}\right) - \Phi\!\left(\frac{\ln(k-\frac{1}{2})-\mu}{\sigma}\right)}{1 - \Phi\!\left(\frac{\ln\frac{1}{2}-\mu}{\sigma}\right)}, \tag{4}$$

$$P_{\mathrm{pl}}(k) = \frac{k^{-\alpha}}{\zeta(\alpha)}, \tag{5}$$

where Φ is the standard normal distribution function and ζ the Riemann zeta function. The geometric model (2) is the distribution of gap lengths whenever the loss process has a single loss state with a constant chance of ending in each slot: independent (Bernoulli) losses, and the two-state Gilbert–Elliott channel, whose losses are correlated in time but whose loss periods still have geometric lengths. It is this single-loss-state assumption on gap lengths that is tested here. The two-geometric mixture (3) is equivalent to a Markov chain with one up state and two loss states ("short loss" and "long gap"); (4) is a lognormal discretised to integer lengths; and (5) is a discrete power law. Models are compared by Akaike's information criterion, $\mathrm{AIC} = 2p - 2\ln\hat{\mathcal{L}}$ for p parameters and maximised likelihood $\hat{\mathcal{L}}$ [45], lower being better. Because AIC only ranks models, each is also tested absolutely, by a discrete Kolmogorov–Smirnov statistic $D = \max_k |\hat{F}(k) - F(k)|$ with a parametric-bootstrap p-value (200 samples from the fitted

model, each refitted). Dependence between consecutive gaps and up-runs is measured by Spearman's rank correlation.

D. Energy model and loss-handling strategies

The node is a Libelium Waspmote-class host [46] with one of three radios: NB-IoT (the N211 module measured in [34], with release assistance, RAI), LoRaWAN at spreading factor 7 (SF7), or LoRaWAN at SF12 in the EU868 band. Each transmission attempt a is charged the measured energy e_a of its radio and outcome:

NB-IoT. The measurements [34] send UDP datagrams of 12–512 B in normal coverage to a server that echoes them, without an application protocol such as CoAP. An unconfirmed report costs 0.31 J (connection released after the uplink) and a confirmed one 0.33 J (released after the server's echo, which is the acknowledgement: it comes from the application, end to end). A bulk frame of up to 512 B, the module's largest datagram, costs up to 60% more than a 20-byte report [34]. No published measurement was found of the energy that an NB-IoT modem spends searching for a cell when there is no network; it is therefore modelled as the receive power of 0.222 W [35] times a search time T_{search}, swept over $\{2.2, 10, 60\}$ s (default 10 s, 2.22 J per failed attempt).

LoRaWAN. Energies come from the state-by-state current profile of [33], with airtime computed from the payload. At SF7 the acknowledgement arrives in the first receive window, so a confirmed success (0.164 J for a 20-byte report) is cheaper than an unconfirmed report (0.244 J), which keeps both windows open; at SF12 it arrives in the second window and a confirmed success is dearer (0.858 vs 0.729 J). A failed confirmed attempt adds the retransmission time-out, RETRANSMIT_TIMEOUT, of 1–3 s [47]. The number of transmissions depends on the specification version. LoRaWAN 1.0.2 strongly recommends eight transmissions of a confirmed frame, with the data rate lowered every two [48]; LoRaWAN 1.0.4 transmits a frame NbTrans times, a number between 1 and 15 set by the network, with no change of data rate [49]. Both are evaluated. The acknowledgement of a confirmed uplink comes from the network server; Sec. VII-A returns to what this means for the design. Frames carry at most the regional maximum payload [47], and the 1% duty cycle forbids a new frame before 100 times the previous frame's airtime has passed.

A reading in a bulk frame takes 12 B (four parameters, a time offset and a sequence number) and a frame 4 B of overhead, so a frame holds 42 readings on NB-IoT, 18 at SF7 and 3 at SF12. The host is awake for every attempt, at $P_{\mathrm{on}} = 69$ mW [46], and its awake time is added to e_a. Over a record of D days with sampling interval T_{s} (s), the transmission energy per day and the modelled lifetime on a battery of energy B are

$$E_{\mathrm{tx}} = \frac{1}{D}\sum_a e_a, \qquad Y = \frac{B}{365\,(E_{\mathrm{floor}} + E_{\mathrm{tx}})}, \tag{6}$$

$$E_{\mathrm{floor}} = 86{,}400\,(P^{\mathrm{radio}}_{\mathrm{sleep}} + P^{\mathrm{host}}_{\mathrm{sleep}}) + \frac{86{,}400}{T_{\mathrm{s}}}\,P_{\mathrm{on}}\,T_{\mathrm{sense}}, \tag{7}$$

where the floor (7) covers sleep and sensing: radio sleep power 9.35 μW for NB-IoT in power-saving mode [34] and 148 μW for the LoRa radio [33], host deep sleep 0.12 mW [46], and an assumed $T_{\mathrm{sense}} = 5$ s awake per sample for probe settling and reading, with a 6,600 mAh battery at 3.7 V ($B = 87.9$ kJ). Absolute energy depends strongly on the platform and the protocol stack: an independent measurement on a different board and stack [37] (a 44-byte uplink, TCP without RAI on NB-IoT) reports 3× the per-message energy of this model for LoRaWAN SF7 and 40× for NB-IoT. The results are therefore stated as ratios between strategies on the same radio, which Sec. V-F tests against wide parameter uncertainty; lifetimes are illustrative.

The following strategies are compared:

- *Unconfirmed*: one unconfirmed attempt per reading; a lost reading is never recovered.
- *Unconfirmed + back-off* (NB-IoT only): as above, but when the modem reports that there is no network the node backs off its attempts as in (8) and discards the readings it cannot send. This is the fair NB-IoT baseline: a node that searches for a cell in every slot of a day-long gap wastes energy for no benefit.
- *ARQ*: confirmed, up to 8 transmissions per reading on LoRaWAN or 3 on NB-IoT, with 3 on LoRaWAN as a sensitivity.
- *Confirmed + buffer + back-off* ("ARQ + buffer"): a reading that fails is buffered and the node enters outage mode, in which it sends one small confirmed probe at a time. After the n-th failed probe it waits

$$w_n = \min\left(2^n,\, K\right)\ \text{slots}, \qquad K = 6\ \text{h}, \tag{8}$$

  before the next, buffering the readings of the skipped slots without radio energy. When a probe succeeds, the buffer is uploaded in confirmed bulk frames. The cap K is examined in Sec. V-D.
- *Batching*: the node always buffers and uploads its buffer every 1 h or 6 h in confirmed bulk frames, with the back-off (8) when an upload fails.

Replay. Delivery records are replayed as the link state: a slot in which the record delivered nothing is a slot in which the first attempt fails. What happens to later attempts in the same slot is not in the record. Three models bracket it. In the *full-slot* model the link stays down for the whole slot, so every retry fails (pessimistic for ARQ). In the *restoration* model the link returns at a uniformly random time between the last missing and the next delivered instant of each gap, so only a gap's last slot can gain from a retry. In the *burst* model, in addition, gaps of up to 1 h are read as radio-loss bursts, in which every retry fails with the probability of a loss following a loss on LoED (0.25), rather than surely (optimistic for ARQ). Retries happen at their real times: after the receive windows, the retransmission time-out and the duty-cycle wait. Independent radio losses can be added per attempt, with probability $p_{\mathrm{radio}} = 0.096$ from LoED (Sec. IV-C). The buffer is unlimited, writing to it costs no energy, and a backlog is uploaded at once unless stated otherwise; Sec. V-C relaxes all three, with flash-write energy from a datasheet [50]. The reported metrics are the energy (6), the share of readings delivered, and the share of all readings that reached the server within 1 h.

TABLE III
MODEL INPUTS: MEASURED (M) IN THE CITED WORK, INFERRED (I) FROM THE RECORDS, OR ASSUMED (A); WHERE ASSUMED, HOW IT IS TESTED.

Input	Value		Tested by
Link state	from the delivery records	I	gap cause, timing models
Retry timing in a gap	full slot / restoration / burst	A	all three
Reading generated in a gap	yes	A	node-off share 0–50%
Radio loss	0 or 0.096 per attempt (LoED)	I	both
Cadence	611 s, 32.5 min, 5–60 min (USGS)	I	–
Payload	20 B report; 12 B per reading + 4 B	A	–
ACK layer	LoRaWAN: network server; NB-IoT: application	M	application ACK variant
Transmissions	8 (LoRaWAN 1.0.2), NbTrans 3 or 8 (1.0.4); NB-IoT 3	M	both versions
LoRaWAN energies	state currents [33]	M	$\times$0.5–2
NB-IoT energies	per report [34]	M	$\times$0.5–2
NB-IoT search	0.222 W $\times$ 2.2–60 s	A	sweep
Host	69 mW awake, 0.12 mW asleep [46]	M	$\times$0.5–2
Sensing	5 s awake per sample	A	ratios unaffected
Storage	unlimited, free	A	RAM 2–64 KB, flash
Backlog drain	at once	A	duty-cycle limit
Back-off cap K	6 h	A	1–24 h
Battery	6,600 mAh, ideal	A	ratios only

Table III lists every model input and whether it was measured, inferred from data, or assumed.

E. The server's real-time picture and the tolerances

Between deliveries the server must show something. Let $x(t)$ be the true reading of one parameter at slot t, $A(\tau)$ the set of slots whose readings have arrived at the server by time τ, and $s^*(\tau) = \max A(\tau)$ the latest of them. The server shows

$$\hat{x}(\tau) = \begin{cases} x(\tau), & \tau \in A(\tau), \\ x(s^*) + \Delta(\tau), & \text{otherwise}, \end{cases} \tag{9}$$

where the diel correction $\Delta(\tau) = x(\tau - 24\,\mathrm{h}) - x(s^* - 24\,\mathrm{h})$ if both of these readings are in $A(\tau)$, and $\Delta(\tau) = 0$ otherwise, that is, the latest arrived reading plus the site's own change over the same hours one day earlier. This *own-diel* estimator is the best simple real-time estimator in the river data used here (Sec. VI). The picture is scored as the share of slots at which $|\hat{x}(\tau) - x(\tau)| \le \delta$, for a tolerance δ per parameter, the inequality being inclusive. Three tolerance levels are used. At the reference level L2, δ is 0.2 °C for T, 2% saturation for DO, 0.1 for pH and 2% of the record's median for EC; the tight level L1 halves these (0.1 °C, 1%sat, 0.05, 1%), and the loose level L3 widens them (0.5 °C, 5%sat, 0.2, 5%). L2 lies at or inside the U.S. Geological Survey's rating of an "excellent" continuous record [51] for every parameter: over the sites used here, the median L2 tolerance is 0.20 °C against 0.20 °C for T, 2.0 against 4.7 %sat for DO (0.3 mg/L or 5% of the reading), 4.2 against 6.3 μS/cm for EC (3% of the reading; USGS rates conductance in percent, as the EC tolerance is defined here) and 0.10 against 0.20 for pH. An estimate within L2 is thus no further from the truth than the sensor's own rated accuracy. All three levels are small compared with the median daily range at these sites (T 1.10 °C, DO 10.0 %sat, EC 12.0 μS/cm, pH 0.25).

F. River water-quality data

River signals come from the U.S. Geological Survey's National Water Information System [52]: instantaneous values of T, EC, DO and pH, approved values only. DO concentrations are converted to percent saturation with the Benson–Krause equation [53] at the measured water temperature. Sec. VI uses 2023 data from 9 river chains, sets of gauging sites on the same river (27 sites). Sec. VII uses every downloaded site that has all four parameters in complete cycles for at least half of the year: 30 of 39 sites in 2023, the development year, and 33 of the same candidates in 2024, the held-out year.

IV. How the Records Lose Data

A. Water-quality deployments: most missing data are in long gaps

Fig. 1 shows when each record was down. Two features are visible at once: many short gaps, and a few gaps lasting days.

InWaterSense node 1 had 1,619 delivery gaps. Many were short: 47.3% lasted a single slot, but together they held only 4.7% of the missing readings. At the other end, 14 gaps of a day or more (0.9% of all gaps) held 50.7% of the missing readings (bootstrap 95% CI 29.4%–64.6%), and gaps of 6 h or more held 68.5% (54.9%–77.2%). The longest lasted 13.3 days. Fig. 2 shows the whole curve.

The Weija deployment, on a different continent, shows the same structure. Its conductivity stream had 42 gaps and lost 15.4% of its readings; 82.9% of the missing readings fell in gaps of a day or more, and the share was between 71.0% and 100.0% across all 9 streams, the longest gap lasting 36.7 days. With only 3 day-long gaps in the conductivity stream, its confidence interval is wide (0.0%–93.5%), so Weija is treated as qualitative support for the InWaterSense result rather than as a second estimate of it. As Sec. III-B showed, Weija's long gaps are silences of the whole node, and most of InWaterSense's long gaps were shared with the second node behind the same gateway.

B. The geometric gap-length model fails, and no tested model fits

Table IV compares the four gap-length models (2)–(5). The geometric model (2), whose single loss state underlies both independent losses and the Gilbert–Elliott channel, ranks last on every record, by 3,677 AIC points on InWaterSense, 120 on Weija and 43,915 on UVA. Its failure is in exactly the quantity that matters, the long-gap share (1): fitted to InWaterSense, it puts 11.4% of the missing readings in gaps of 6 h or more, against 68.5% observed, and less than 0.001% in gaps of a day or more, against 50.7% (Fig. 2). On Weija it puts 36.9% in day-long gaps, against 82.9%. AIC only ranks the models, so each was also tested absolutely, by the bootstrap Kolmogorov–Smirnov test of Sec. III-C. The geometric model is rejected on every record ($p \le 0.005$, the smallest value the 200 bootstrap samples can give). With 1,619 gaps on InWaterSense and 36,519 runs on UVA, every tested model is rejected, but by very different margins: the geometric model's largest distance between fitted and observed distribution functions is 0.48 on

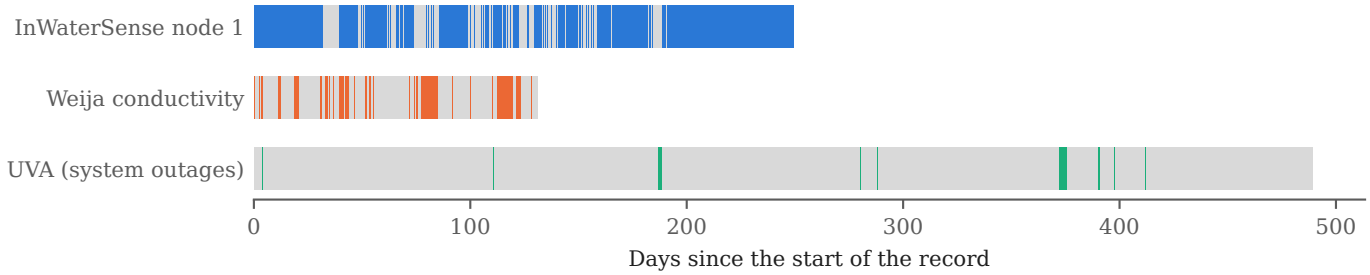


Fig. 1. Delivery timelines of the three records with long gaps: coloured spans are slots in which nothing was delivered (UVA: system outages only; its short radio losses are not shown). Each record is drawn on its own time axis, from its first slot.

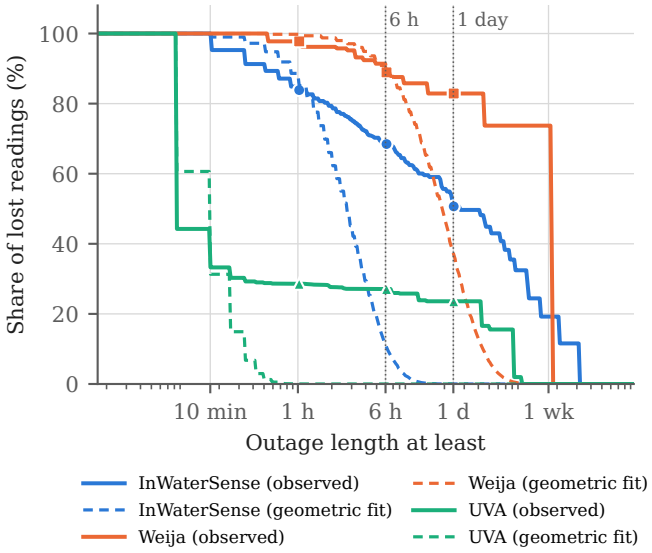


Fig. 2. Share of missing readings that fall in gaps at least as long as the value on the horizontal axis, $S(L)$ in (1) (solid), and the same share predicted by the geometric model (2) fitted to each record's gap lengths (dashed). Markers at 1 h, 6 h and 1 day. InWaterSense node 1 (611-s slots), Weija conductivity (32.5-min slots) and UVA (all sensors, 5-min slots; radio and system losses together).

InWaterSense and 0.26 on UVA, against 0.027 and 0.008 for the best of the other models. On Weija's few gaps, the lognormal and power-law models are not rejected ($p = 0.37$ and 0.64). The ranking is not driven by one extreme gap: without the longest gap of each record, every record keeps its best model. A geometric gap-length model is thus strongly unsupported relative to the alternatives tested, and no tested model describes the larger records exactly.

No single alternative wins everywhere. The discrete lognormal is best by AIC on InWaterSense, the discrete power law on Weija (with the lognormal 1.7 AIC points behind), and the two-geometric mixture on UVA. The mixture's form, many short losses and a few long gaps, is the common thread, and the records differ visibly in how many long gaps they have (Fig. 2). Gaps are also not independent of each other. On InWaterSense a long gap tends to be followed by another long gap (Spearman $\rho = +0.27$) and by a short up-run ($\rho = -0.20$; both $p < 0.001$), so gaps cluster into bad periods that a renewal model, drawing each gap independently, cannot reproduce. On Weija, with 42 gaps, neither correlation is distinguishable from zero ($\rho = +0.02$, $p = 0.88$; $\rho = -0.06$, $p = 0.73$). For these reasons all simulations in this paper replay the real records rather than a fitted model.

TABLE IV
AIC OF GAP-LENGTH MODELS (LOWER IS BETTER; BEST PER RECORD IN BOLD). UVA: ALL MISSING-FRAME RUNS OF ALL SENSORS, 5-MIN SLOTS.

Model	InWaterSense	Weija	UVA
Geometric (memoryless)	10,537	341	76,881
Two-geometric mixture	7,107	236	**32,966**
Discrete lognormal	**6,859**	222	33,327
Discrete power law	6,863	**221**	33,180

C. LoRaWAN networks: short radio losses and rare system outages

The two LoRaWAN records separate the causes of loss. On LoED, well-heard devices lost 9.6% of their frames to the radio. Losses were mildly bursty: the probability of losing a frame after a lost frame was 0.25, about 2.6 times the marginal rate. But the bursts were short: of 56,276 radio-loss bursts, 84% were a single frame and 99% at most five frames. Losses also did not coincide across devices beyond what their density implies: a lost frame coincided with a loss of another device at a shared gateway within the same minute in 19.2% of cases, against 18.2% (16.7%–19.5%) under a random time-shift baseline.

UVA adds what LoED, recorded in campaign windows, cannot show: 16 months of continuous operation. The system as a whole was available 98.72% of the time, with 12 system outages (median 1.67 h, longest 82.2 h). Of the 58,211 missing frames, 71.6% were radio losses and 28.4% system losses. The radio regime looks like LoED's: 89% of the 36,519 runs of missing frames were a single frame. The system regime looks like the water-quality deployments': 83% of the system downtime fell in outages of a day or more. With few system outages, UVA's long outages hold a smaller share of its missing frames than at InWaterSense (27.1% in runs of 6 h or more, 23.6% in runs of a day or more; Fig. 2). The classification is not an artefact of the 10-minute bin: with bins of 5 to 30 min the

number of system outages ranges from 14 to 8 (short ones merge), the share of system down-time in outages of a day or more from 82% to 84%, and the system share of missing frames from 28.0% to 28.6%.

Two further observations matter for design. First, *gateway diversity masks gateway outages*: one of UVA's gateways was down for 29.9 days in a row, yet none of the missing frames was a gateway loss, because every sensor was also heard by a second gateway. Second, rain had no detectable effect on link loss: over 18 sensor–gateway pairs, loss was 9.18% in dry hours and 9.21% in rainy hours (Wilcoxon signed-rank test [54], $p = 0.64$).

In answer to RQ1: across the four records studied, losses separate into short radio losses, which are nearly independent, and rarer long delivery gaps or system outages of a gateway, its backhaul, the server or the node. In the two water-quality deployments, gaps of 6 h or more hold most of the missing data (68.5% on InWaterSense, 88.9% on Weija conductivity). A geometric gap-length model misses the second regime almost entirely.

V. Retransmit or Backfill?

The InWaterSense and Weija conductivity records are replayed as the link state through the energy model of Sec. III-D, for NB-IoT (search times T_{search} of 2.2, 10, 60 s), LoRaWAN SF7 and SF12, without and with independent radio losses ($p_{\mathrm{radio}} = 0$ and 0.096): 20 configurations in all, first under the full-slot model, then under the other timing models and the other assumptions of Table III. Table V shows the InWaterSense record; ranges in the text are over all applicable configurations. (Mapped onto the simulator's regular slot grid, the InWaterSense record loses 46.1% of the readings, against 46.0% counted directly.)

A. ARQ pays for losses it can hardly recover

Under the full-slot model, ARQ delivers exactly what unconfirmed reporting delivers: every reading lost to a gap is lost after all of its retries. On LoRaWAN it costs 2.59×–7.35× the energy of unconfirmed reporting with 8 transmissions, and 1.51×–2.94× with 3. On InWaterSense at SF7 this is 447.1 J/day instead of 61.6 J/day, a modelled lifetime of 0.46 instead of 1.80 years. With independent radio losses added, ARQ recovers them, raising the delivered share by $+5.0$ to $+7.9$ percentage points.

The full-slot model makes "ARQ recovers nothing" partly true by construction, so it is only the pessimistic end of a bracket. If the link may return at any moment of a gap's last slot (restoration model), retries recover 0.1 to 4.6 percentage points of InWaterSense's readings on LoRaWAN, more at SF12, whose retries span minutes. If, in addition, every gap of up to an hour is a radio-loss burst through which retries usually pass (burst model, optimistic for ARQ), they recover 6.5 to 7.7 points. Against the 46 points that unconfirmed reporting loses on this record, ARQ thus recovers under a fifth in every model, at 2.3–7.3 the energy of unconfirmed reporting; on Weija it recovers at most 0.9 points. NB-IoT ARQ recovers nothing in any model, because the modem reports that there is no network and the node stops retrying. The specification version changes the price little: with eight transmissions at a fixed data rate (LoRaWAN 1.0.4, NbTrans = 8) ARQ costs 6.8× the energy of unconfirmed reporting on InWaterSense at SF7, against 7.3× with the data-rate stepping of LoRaWAN 1.0.2; at SF12 there is no lower rate to step to. Retransmission is the right tool for the radio regime and the wrong one for the long gaps, which hold most of the missing data.

B. On NB-IoT, back-off is where the energy is won

An NB-IoT node that tries to send in every slot of a gap searches for a cell each time. Searching in every slot costs 1.20×–6.53× the energy of backing off (unconfirmed reporting without back-off against unconfirmed reporting with it), the ratio growing with the share of time in gaps and with T_{search}. Because no published measurement of T_{search} was found, this is a sweep, not an estimate; with back-off, however, a node in a long gap searches at most once every 6 h instead of once per slot. The fair NB-IoT baseline is therefore unconfirmed reporting *with* back-off, not without it.

C. Buffering buys completeness at a modest energy cost

Confirmed reporting with a buffer and back-off delivers every generated reading in every configuration. Relative to the fair baseline that recovers nothing, its energy changes by -5.5% to $+15.3\%$ on NB-IoT and by -14.6% to $+25.6\%$ on LoRaWAN. The sign depends on the radio. At SF7 buffering ranges from -14.6% to $+0.6\%$, because a confirmed success, with its acknowledgement in the first receive window, costs less than an unconfirmed report (Sec. III-D), and because the node stops transmitting into gaps. At SF12 it is always dearer, by $+7.8\%$ to $+25.6\%$, because the acknowledgement arrives in the second window. On InWaterSense at SF7 it uses 52.7 J/day (1.93 years) and delivers all readings, against 61.6 J/day and 1.80 years for unconfirmed reporting, which loses 46.1% of them (Table V). The other timing models do not change this: buffering's energy stays within 0.67–1.08 of unconfirmed reporting on LoRaWAN in all three.

This completeness rests on three assumptions, each tested here.

The reading must exist. Buffering cannot deliver a reading that a switched-off node never took. When up to half of the slots in long gaps are reassigned to a node that was off (Sec. III-B), buffering still delivers every reading generated, but only 83% of the expected readings on InWaterSense and 92% on Weija; its energy relative to the fair baseline moves from 0.85 to 1.02 (SF7, InWaterSense). Weija has few long gaps, so its realised node-off shares are coarse (0.49 for a nominal 0.1).

The buffer must hold the backlog. The longest backlog on these records is 22 KB of readings. A buffer in 2 KB of RAM loses 24% of InWaterSense's readings, 8 KB loses 10%, and 64 KB loses none. Writing every reading to a serial flash memory [50], with the host awake, costs at most 0.03 J/day in the worst case of the datasheet, against at least 9 J/day of radio energy: storage energy is negligible.

TABLE V
LOSS-HANDLING STRATEGIES ON THE INWATERSENSE RECORD (FULL-SLOT MODEL, NO ADDED RADIO LOSS): TRANSMISSION ENERGY PER DAY, E_{tx} IN (6), FOR EACH RADIO (NB-IOT: N211 WITH RAI, $T_{\mathrm{search}} = 10$ S), AND THE SHARE OF READINGS DELIVERED AND DELIVERED WITHIN 1 H (LORAWAN SF7; NB-IOT FOR THE NB-IOT-ONLY STRATEGY). ARQ: LORAWAN 1.0.2 BEHAVIOUR (DATA RATE STEPPED DOWN EVERY TWO TRANSMISSIONS). BACK-OFF AS IN (8), CAPPED AT 6 H. –: NOT APPLICABLE.

	Energy (J/day)			Readings	
Strategy	NB-IoT	SF7	SF12	delivered	within 1 h
Unconfirmed, every reading	229.9	61.6	147.2	53.9%	53.9%
Unconfirmed + back-off (NB-IoT only)	59.5	–	–	47.1%	47.1%
ARQ (8 attempts LoRaWAN; 3 NB-IoT)	231.5	447.1	796.1	53.9%	53.9%
ARQ, 3 attempts (LoRaWAN)	–	175.5	354.3	53.9%	53.9%
Confirmed + buffer + back-off	63.8	52.7	158.7	100.0%	52.8%
Hourly batch + buffer + back-off	23.6	18.9	89.5	100.0%	52.0%
6-hourly batch + buffer + back-off	13.3	11.3	79.0	100.0%	11.2%

The backlog must be drained without starving live traffic. If the backlog may use only what the 1% duty cycle leaves after one live frame per slot, every reading is still delivered. At SF7 the longest backlog clears in 64 h. At SF12, where one bulk frame fits per slot, the median backlog clears in 1.2 h, but in InWaterSense's worst period, when new gaps keep interrupting the drain, a backlog persists for 63 days, and readings generated while it drains reach the server up to 25 days late (95th percentile). Backfill at SF12 after long gaps should therefore interleave live readings ahead of the backlog, as SensorScope's operators learned [6].

D. Batching is cheaper, and costs little timeliness

Bulk frames carry many readings for little more than the cost of one (Sec. III-D). Hourly batching with buffer and back-off delivers every reading and uses 1.33×–3.33× less energy than the fair per-reading baseline, 18.9 J/day (2.65 years) on InWaterSense at SF7. Its share of readings that reach the server within 1 h differs from unconfirmed reporting's by only -3.3 to $+7.3$ percentage points, because under long gaps the gaps, not the batching interval, decide how many readings arrive within the hour. Six-hourly batching is cheaper still, but delivers only a small share of readings within the hour (Table V).

The back-off cap K trades energy against how long the server waits once the network is back. On InWaterSense at SF7, probing in every slot costs 2.2× the energy of $K = 6$ h (3.7× on NB-IoT, where every probe into a dead network searches for a cell). Raising K from 6 h to 24 h saves only a further 12%, while the 90th percentile of the delay between the link's return and the arrival of the gap's last reading grows from 20 h to 61 h. The chosen $K = 6$ h sits at this knee.

The price of batching is the timeliness of individual readings: a reading can wait up to an hour, and an hour can matter. It matters for alarms, when something happens in the river, and for the server's real-time picture of fast-changing parameters. Sec. VII addresses both.

E. Many nodes recovering at once

The analysis so far is per node. A system outage, however, ends for every node behind the gateway at once, and their backlogs then compete for the same channels and for the gateway's acknowledgements. A discrete-event model of one EU868 gateway and N nodes that buffered through a common outage captures this (Table VI). Uplinks on the same channel and spreading factor collide unless one is 6 dB stronger; the gateway demodulates at most eight at once and cannot receive while it transmits; it acknowledges in the first receive window if its 1% sub-band allows, else in the second (869.525 MHz, SF12, 10% sub-band), else not at all; nodes back off as in (8), drain with up to eight transmissions per frame, and respect their own duty cycle. The collision rule reproduces the pure-ALOHA success probability e^{-2G} in a test.

Without randomisation, the deterministic back-off of (8) keeps nodes that entered an outage together synchronised: from 50 nodes at SF7, almost none recovers within 72 h, because they collide at every probe. Delaying each probe by a random time of up to an hour removes this. At SF7 every node then recovers, up to 500 per gateway: the median node clears its backlog within 2–9 h of the end of the outage and the last within 42 h, at up to 12.5× the energy of a node recovering alone. At SF12 the gateway's acknowledgements, not the uplinks, run out: its first-window sub-band reaches 1.0% airtime, its 1% limit, up to 29% of the frames it receives go unacknowledged and are sent again, and after a one-day outage only 93% of 100 nodes recover within 72 h. The per-node results above hold for cells of tens of nodes; larger cells of SF12 nodes need randomised recovery, and fewer acknowledgements per reading, for example one confirmed frame per backlog chunk.

F. How robust are the energy ratios?

The energy model cannot be checked against hardware here, and absolute per-message energy differs severalfold between platforms (Sec. III-D). The ratios were therefore recomputed 100 times with every measured component multiplied by an independent factor between 1/2 and 2 (log-uniform): unconfirmed and confirmed LoRaWAN reports, the acknowledgement time-out, the NB-IoT reports, bulk-size slope and search power, and the host's awake power, with the NB-IoT search time drawn between 2.2 and 60 s. On LoRaWAN, ARQ costs more than unconfirmed reporting in 100% of the draws, at least 1.8× in 97.5% of them; on NB-IoT, where ARQ

TABLE VI
N NODES RECOVERING FROM A COMMON ONE-DAY OUTAGE BEHIND ONE GATEWAY (PROBE JITTER UP TO 1 H; MEAN OF 3 RUNS; 72 H HORIZON). CLEARED: NODES WHOSE BACKLOG WAS DELIVERED; MEDIAN AND LAST CLEARANCE (H AFTER THE OUTAGE); PDR: UPLINKS RECEIVED AND ACKNOWLEDGED; COLLIDED; RECEIVED BUT UNACKNOWLEDGED; NODE ENERGY OVER THE HORIZON RELATIVE TO A NODE RECOVERING ALONE.

	N	Cleared	Med. (h)	Last (h)	PDR	Coll.	No ACK	Energy
SF7	10	100%	5	6	99%	0%	1%	1.1×
SF7	50	100%	5	10	97%	1%	1%	1.5×
SF7	100	100%	5	13	93%	1%	3%	1.9×
SF7	250	100%	6	21	85%	2%	9%	3.3×
SF7	500	100%	8	42	77%	2%	15%	7.1×
SF12	10	100%	8	14	92%	2%	2%	1.3×
SF12	50	100%	15	43	67%	7%	16%	3.7×
SF12	100	93%	26	>72	47%	13%	25%	7.2×
SF12	250	36%	>72	>72	38%	18%	28%	4.8×
SF12	500	13%	>72	>72	32%	21%	29%	3.1×

stops retrying when there is no network, it costs about the same (median 1.01×). Hourly batching is cheaper than the fair baseline in at least 98% of the draws at SF7 and on NB-IoT, but only in 85%–93% at SF12, where it can cost up to 1.24× as much: with few readings per frame and the acknowledgement in the second window, its advantage depends on the energy of a confirmed SF12 frame. Buffering's energy relative to the fair baseline has a median of 0.81×–1.10× over the draws, but ranges from 0.50× to 2.08×: its sign is set by the ratio of confirmed to unconfirmed energy, the mechanism of Sec. V-C, and only measurement on the target hardware can settle it. The orderings that the design rests on, ARQ above unconfirmed reporting on LoRaWAN and batching below per-reading reporting at SF7 and on NB-IoT, survive this uncertainty; the size of buffering's premium does not.

In answer to RQ2, ARQ is the wrong remedy for long gaps: under any of the timing models it recovers only the short radio losses, and on LoRaWAN it multiplies energy. Buffering with back-off recovers every reading the node generated, given tens of kilobytes of storage, at an energy close to that of recovering nothing in the model (-14.6% to $+25.6\%$ on LoRaWAN, -5.5% to $+15.3\%$ on NB-IoT), and between 0.50× and 2.08× of it under a factor-of-two uncertainty in every energy component. Hourly batching is the energy-efficient way to do it at SF7 and on NB-IoT, at a timeliness cost that only alarms and fast parameters notice. Many nodes recovering at once need randomised probes, and large SF12 cells are limited by the gateway's acknowledgements.

VI. CAN RECONSTRUCTION REPLACE DELIVERY?

If a server could estimate the missing readings well enough, it would not need them delivered. This is tested on 9 river chains: sets of USGS gauging sites along the same river (Delaware, Illinois, Tualatin, Clackamas, McKenzie, Monongahela, Youghiogheny, Schuylkill and Klamath; 27 sites in all), each pair of sites on a chain serving as target and neighbour in both orders (53–57 ordered pairs per parameter).

A. Method

All series are placed on a common 1-h grid. For every ordered pair, parameter and gap length (1, 3, 6, 12, 24, 48, 72 h), gaps are simulated at the target (a median of 300 per pair, parameter and length), drawn from the last 80% of the year. A gap covers the slots after the last observed slot t_0 and before the next observed slot t_1. Let y be the neighbour's series, lagged by ℓ hours, and b a scaling slope; $\ell \in [0, 48]$ h is chosen to maximise the correlation between 6-h changes of target and lagged neighbour in the first 20% of the year, and b is the least-squares slope of those changes (pairs whose best correlation is negative are dropped). *Real-time* estimators use only data up to t_0, and the neighbour during the gap:

$$\hat{x}_{\mathrm{pers}}(t) = x(t_0), \tag{10}$$

$$\hat{x}_{\mathrm{diel}}(t) = x(t_0) + x(t - 24\,\mathrm{h}) - x(t_0 - 24\,\mathrm{h}), \tag{11}$$

$$\hat{x}_{\mathrm{nb}}(t) = x(t_0) + b\,[y(t) - y(t_0)], \tag{12}$$

$$\hat{x}_{\mathrm{dn}}(t) = \hat{x}_{\mathrm{diel}}(t) + b\,[y(t) - y(t_0)] - b\,[y(t - 24\,\mathrm{h}) - y(t_0 - 24\,\mathrm{h})]. \tag{13}$$

Persistence (10) holds the last value; own diel (11) is the server's estimator (9); neighbour change (12) adds the neighbour's scaled change since the gap began; and diel + neighbour (13) adds only the part of that change which the neighbour's own diel pattern does not explain. *Bridging* estimators also know $x(t_1)$, as they would once backfilled data arrive:

$$\hat{x}_{\mathrm{int}}(t) = x(t_0) + \frac{t - t_0}{t_1 - t_0}\,[x(t_1) - x(t_0)], \tag{14}$$

$$\hat{x}_{\mathrm{in}}(t) = \hat{x}_{\mathrm{int}}(t) + b\,[y(t) - \bar{y}(t)], \tag{15}$$

where $\bar{y}$ is the straight line from $y(t_0)$ to $y(t_1)$, so that (15) adds the neighbour's departure from its own line to the interpolation (14). The error of an estimator on a gap is its maximum absolute error over the gap, $\max_{t_0<t<t_1} |\hat{x}(t) - x(t)|$; the median over gaps is reported, together with the share of gaps whose maximum error is within the L2 tolerance. Ordered pairs are not independent: a site appears in several pairs, and the pairs sit in 9 chains. Confidence intervals therefore resample whole chains, and pairs within each drawn chain (5,000 draws).

B. Results

Table VII gives the results for 1-h and day-long gaps. The text reports 24-h gaps; relative gains are medians over pairs, and exclude the 3 pH pairs whose target (site 14206694) has a median error of exactly zero, for which a relative gain is undefined.

In real time, a site's own diel pattern is the estimator to beat. It beats persistence in 77% of pairs for DO and 84% for T. A neighbour improves on the better of the two own-data estimators in only 30% of pairs for DO (median gain -8.8%), 9% for pH (-6.6%) and 23% for EC (-1.9%). Temperature is the exception: a neighbour helps in 72% of pairs (median gain $+6.5\%$), consistent with a shared, non-periodic weather driver.

When bridging a gap after the data return, the neighbour helps more. The neighbour's shape improves interpolation in

TABLE VII
RECONSTRUCTION ON 9 RIVER CHAINS: MEDIAN OVER ORDERED SITE PAIRS OF THE MEDIAN MAXIMUM ABSOLUTE ERROR OVER A GAP (T IN °C, DO IN %SAT, PH IN PH UNITS, EC IN μS/CM), AND THE SHARE OF PAIRS IN WHICH THE NEIGHBOUR IMPROVES ON THE TARGET'S OWN DATA. REAL TIME: BEST NEIGHBOUR METHOD, (12) OR (13), VS THE BETTER OF PERSISTENCE (10) AND OWN DIEL (11). BRIDGING: INTERPOLATION + NEIGHBOUR (15) VS INTERPOLATION (14).

		Real time				Bridging		
Param.	Gap (h)	Persistence	Own diel	Best neighbour	Neighbour wins	Interpolation	Interp. + neighbour	Neighbour wins
T	1	0.07	0.05	0.05	23%	0.03	0.03	19%
T	24	0.97	0.70	0.65	72%	0.80	0.56	93%
DO	1	0.67	0.49	0.58	19%	0.26	0.34	13%
DO	24	9.07	4.33	5.13	30%	7.75	6.27	70%
pH	1	0.000	0.025	0.018	7%	0.013	0.013	9%
pH	24	0.175	0.125	0.149	9%	0.172	0.172	38%
EC	1	0.50	0.50	0.66	16%	0.25	0.27	16%
EC	24	7.50	8.25	7.50	23%	4.44	4.42	25%

93% of pairs for T (median gain $+23.4\%$) and 70% for DO ($+10.3\%$), but not for pH (38% of pairs, -0.2%) or EC (25%, -2.2%).

None of the six estimators keeps day-long gaps reliably within tolerance. For 1-h gaps, the better own-data real-time estimator keeps a median of 97% of gaps within the L2 tolerance. For 24-h gaps, even the best of all six, including the bridging ones that already know the value after the gap, keeps a median of only 31% of gaps within tolerance (chain-resampled 95% CI 18%–49%; upper quartile of pairs 63%), from 9% for T to 49% for pH.

A naive baseline overstates the value of neighbours. Judged against persistence alone, a neighbour method wins in 77% of pairs for DO and 36% for pH, and the best neighbour method cuts the median DO error from 9.07 to 5.13 %sat. Against the target's own diel pattern, which reaches 4.33 %sat, most of that gain disappears: the neighbour wins in 30% of DO pairs and 9% of pH pairs. Much of what a naive test credits to spatial correlation is the shared day–night cycle, which the target's own history already contains.

The fitted neighbour relation does not follow the flow. The neighbours lie 2.8 to 300 km from their targets (median 42 km). If the neighbour carried water to the target, the selected lag would be positive for upstream neighbours and zero for downstream ones; instead, for DO a downstream neighbour gets a positive lag in 64% of pairs and an upstream one in 68%. The lag that maximises the correlation of 6-h changes tracks shared timing, of daylight and weather, rather than travel time. Evaluations of spatio-temporal reconstruction on river data therefore need a per-site diel baseline, and spatial structure taken from the river network rather than from correlation alone [30], a point also made for graph imputation in general [29].

C. Contemporary deep imputers

The estimators above are deliberately simple. To test whether the conclusion is a limit of reconstruction or of these estimators, two contemporary deep imputers were added: SAITS, a self-attention model [27], and StemGNN, a graph neural network that learns the relations between series [26] (implementations of PyPOTS; GRIN and SPIN need a graph library that the available machine could not hold). Each model sees all four parameters of every site of a chain in 192-h windows, normalised per window and series from the observed values, and is trained only on the first half of 2023 with masks that include whole-site blocks of 1–72 h, the evaluation scenario; the checkpoint with the best block error on a held-back part of the first half is kept. A first configuration, normalised with half-year means and spreads, under-fitted by an order of magnitude even on 1-h gaps and was discarded before any comparison. The models are tested on gaps drawn from the second half of the year, in which a whole site is missing for 1, 6, 24 or 72 h, and the paper's estimators are scored on the same 48,455 gap–parameter cases. When bridging, they also see the data after the gap; in real time, nothing of the target after its start (the neighbours up to the end of the gap, which favours the deep models).

The deep imputers do not change the conclusion. On 1-h gaps they keep 65% (SAITS) and 62% (StemGNN) of the cases within tolerance when bridging, against 97% for interpolation, and 57% and 52% in real time, against 91% for the own-diel estimator. On day-long gaps they keep 25% and 23% when bridging (interpolation: 31%) and 18% and 16% in real time (own diel: 29%). Where they help is the error of DO and pH when bridging a day-long gap: SAITS's median maximum error is 4.3 %sat for DO and 0.126 for pH, against 6.7 %sat and 0.148 for interpolation, while it is worse for temperature and conductance. No estimator, simple or deep, keeps more than 55% of the day-long gaps of any parameter within tolerance.

In answer to RQ3, the reconstruction estimators evaluated, simple or deep, cannot replace delivery. For DO and pH the best real-time estimate is the site's own diel pattern, and neighbours add little; for temperature, and for bridging gaps after the data return, neighbours help, and a deep imputer reduces the DO and pH error. For gaps of a day, which hold most of the missing data in the water-quality records (Sec. IV), none of the estimators keeps the error within tolerance in most gaps. Reconstruction's role is the interim estimate between deliveries, and only delivery restores the data. This is the role

it plays in the design of Sec. VII.

VII. A Reporting Design and its Evaluation

Secs. IV–VI point to a design: buffer and back off through gaps (only delivery restores long gaps), batch routine data (cheap, and little less timely when gaps are long), and show the site's own diel estimate between deliveries. Batching leaves two gaps, alarms and the real-time picture of fast parameters, and the design closes them with two triggers that send a batch early.

A. The design

Event-driven batching runs at the node (Algorithm 1, without the bounded-error trigger). Every reading is buffered. Once an hour the node uploads its buffer in confirmed bulk frames. When the node's event detector raises an alarm, it uploads the buffer at once, alarming reading included; if the node is in an outage, an alarm triggers an immediate probe outside the back-off schedule (without resetting it). A failed upload puts the node into outage mode, with the probe back-off (8).

Fig. 3 shows these behaviours in a short illustrative scenario run through the simulator. Before the outage, the alarm makes the event-driven node upload at once, where plain batching would wait for the end of the hour; the alarm during the outage cannot get through; and after the outage, when the backlog arrives depends on where each strategy is in its back-off (8).

Bounded event-driven batching adds a *bounded-error trigger*. Because every upload is confirmed, the node knows which of its readings have reached the server and when; it can therefore compute, at every slot t, the value $\hat{x}_j(t)$ that the server is showing for each parameter j, by evaluating (9) on the readings it knows to have arrived. It uploads its buffer at once (Algorithm 1) if

$$\max_j \ \frac{|x_j(t) - \hat{x}_j(t)|}{\delta_j} > 1, \tag{16}$$

that is, if any parameter's true reading differs from what the server shows by more than its tolerance δ_j. This is the dual-prediction mechanism of error-bounded report suppression [38], [39], used here to bound the error between batches rather than to suppress reports. Three details matter. First, the trigger is not consulted during an outage: the server's estimate may drift far from the truth during a long gap, and a node that probed every time would waste energy on a dead link; only alarms probe. Second, the node's copy of the server must model *when* readings arrive, not only *whether*: on LoRaWAN the duty cycle spaces the frames of a long upload, so a reading enters $A(t)$ in (9) only after its frame's arrival time, and the newest valid reading goes in the first frame of every upload, so that the value the server most needs arrives first. Third, the node's knowledge of the server comes from acknowledgements, so it is only as good as their boundary. On NB-IoT the confirmation is the application server's reply [34], and the node knows what the server holds. On LoRaWAN a confirmed uplink is acknowledged by the network server; the forwarding to the application server, which UVA's "system" losses show can fail, is not covered. A variant of the trigger closes this gap at a cost (Sec. VII-H): it keeps the bound for both possible server states, with and without the readings the network server acknowledged, until an application acknowledgement returns in the downlink of a later uplink.

Algorithm 1 Bounded event-driven batching: node, one slot

```
append the slot's reading x_t to the buffer
if in outage mode then
    if alarm at t or back-off timer expired then
        probe; on success leave outage mode and upload the buffer
    end if
    return
end if
x̂(t) ← server's estimate (9), from readings whose arrival time is ≤ t
if alarm at t or trigger (16) or hour boundary then
    upload the buffer (newest valid reading first), confirmed bulk frames
    on failure enter outage mode, back-off (8)
end if
```

B. Evaluation protocol

Two phases. The design was developed on 2023 data and then tested on held-out 2024 data. In the development phase, hypotheses H1–H4 and their decision rules were written down before the runs for event-driven batching; after H4 failed, H5–H7 were written for the bounded-error trigger before it was implemented, and H8–H10, on packing order and on the tolerance level, before they were run. Every deviation was recorded when made, with its reason: UVA replayed as system outages plus radio losses (D1); one site excluded because its eligibility had been computed over its record span rather than the year (D2); seeds derived from site identifiers (D3); two implementation defects of the packing order, which made H8 fail as written (D4); and a floating-point tie at the tolerance (D5, found during the revision): pH and T are recorded in 0.1 steps, and an error of exactly one tolerance was counted in or out by rounding noise, in the scores and in the trigger. All development results below are from a rerun with D4 and D5 fixed. For the confirmatory phase, the final design, the analysis code, its decision rules and the eligibility rule were frozen, and the code identified by cryptographic hashes, before any 2024 data were downloaded; the 2024 verdicts below are the output of that frozen code. The plans are kept with the code and hashed; they were not registered with a third party.

Sites. Every USGS site of the 39 candidates with all four parameters in complete cycles for at least half of the year: 30 sites in 2023 and 33 in 2024 (29 in both years). Sampling steps are 5 to 60 min. The first 20% of each record trains the detector; all metrics are computed on the remaining 80%.

Links. The no-outage link, and three real records mapped onto each site's time grid in physical time, wrapped circularly with a random start offset per site (seeded by the site identifier, with a different seed in 2024): InWaterSense and Weija conductivity, in which every missing slot is a down link (the full-slot model, pessimistic for the per-reading strategies), and UVA, replayed as its system outages plus independent radio losses at the rate of a representative sensor (3.08% per

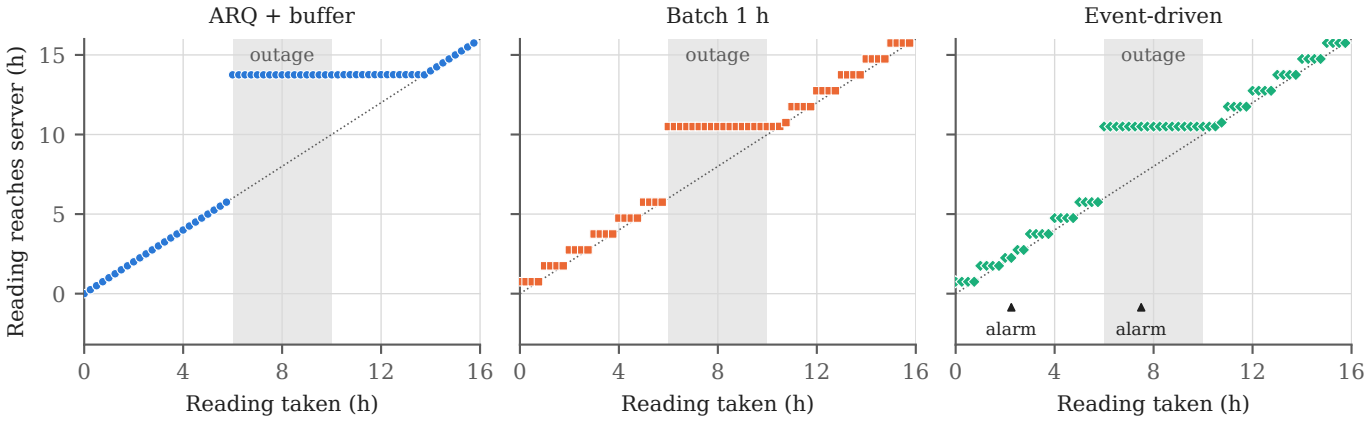


Fig. 3. When each reading reaches the server, in an illustrative scenario run through the simulator: LoRaWAN SF7, 15-min sampling, the link down from 6 to 10 h, and node alarms at 2.25 h (link up) and 7.5 h (link down). Points on the dotted diagonal arrive at once; horizontal runs are readings delivered together. Event-driven batching follows hourly batching except that the alarm at 2.25 h sends its batch early.

attempt). *Radios:* NB-IoT ($T_{\text{search}} = 10$ s), LoRaWAN SF7 and SF12.

Strategies. Unconfirmed, ARQ + buffer, hourly batching ("batch 1 h"), event-driven batching, and bounded event-driven batching (newest-first packing, L2 tolerance unless stated), as defined above and in Sec. III-D.

Injected events. Events are added to the real test-period series, so background dynamics, noise and gaps are real and only the event is synthetic. For parameter j, let $a_j(t)$ be the reading minus its centred 24-h running median, and $r_j(t)$ that anomaly minus its median for the same calendar month and hour of day. The event scale is the robust standard deviation of r_j in the event's month,

$$\sigma_j = \max\left(1.4826\ \mathrm{MAD}[r_j]\,,\ \rho_j\right), \tag{17}$$

floored at the recording resolution ρ_j, estimated from the data as the most common positive difference between sorted distinct values. An event of amplitude $\kappa\sigma_j$, with σ_j from (17), $\kappa \in \{5, 8\}$ and a random sign, lasts 180 min: a *step* adds $\kappa\sigma_j$ for its whole duration, and a *ramp* rises linearly from 0 to $\kappa\sigma_j$; injected values are rounded to ρ_j. Events touch one parameter each, in turn, lie in the test period, at least a day after the start of a data segment and wholly inside it, and at least three days apart (at most 60 per site and setting; 7,548 in 2024). The events are of a realistic size. Their median amplitudes are mild in physical units (5σ/8σ: T 0.74/1.19 °C, DO 7.4/11.9 %sat, EC 7.4/11.9 μS/cm, pH 0.50/0.80), and real 3-hour changes of at least 5σ occur at 100% of the sites for T, 100% for DO, 93% for EC and 60% for pH (8σ: 83%, 97%, 93%, 37%). They are physically plausible: 99.7% of the 2024 events stay within T -0.5–40 °C, positive EC, pH 4–11 and DO 0–250 %sat (99.8% in 2023).

Detector. All strategies use the same causal detector: on the node for the event-driven designs, and on the server, on the readings it has received, for the others. It compares each reading with the reading $m = 120$ min earlier, corrected by a learned hour-of-day profile $p(\cdot)$ of the departure from the running 24-h median, and runs a two-sided CUSUM [55] on the clipped, standardised residual:

$$z_t = \mathrm{clip}\left(\frac{x_t - x_{t-m} - p(h_t) + p(h_{t-m})}{s},\ -c,\ c\right), \tag{18}$$

$$S_t^+ = \max\left(0,\ S_{t-1}^+ + z_t - k\right),$$
$$S_t^- = \max\left(0,\ S_{t-1}^- - z_t - k\right), \tag{19}$$

where h_t is the hour of day and s a robust residual scale floored at ρ_j. An alarm is raised when either sum in (19), driven by the residual (18), exceeds H, with $k = 3$, $H = 8$ and $c = 6$; the detector then stays in an event state until $|z_t| < 2$ for 60 min, and resets its sums. The profile and the scale are initialised on the training period and updated by exponential smoothing (memory about 7 days) only outside the event state. These parameters were fixed before any evaluation and were not tuned on its data. An event counts as detected if an alarm on its parameter occurs during it. In 2024 the detector caught a median (over sites) of 67% of 5σ steps, 91% of 8σ steps, 13% of 5σ ramps and 65% of 8σ ramps (58.2% of all events), with a median of 0.8 false alarms per parameter per 30 days (mean 1.1).

Metrics. Alarm latency: the time from an event's onset to the arrival of its alarm at the server, for detected events, split by whether the link was up at onset. Because an operator cares about the whole chain, the share of *all* injected events that the server knows of within 1, 6 and 24 h is also reported. *Energy*: transmission J/day, E_{tx} in (6), including uploads caused by alarms and false alarms; for the event-driven designs, averaged over the four event settings. *Picture*: the share of scored slots at which the server's real-time picture (9) is within tolerance, on the series without injected events. Each is summarised as the median over sites per link and radio (12 link–radio combinations; 48 with the four parameters), with 95% confidence intervals from resampling sites (10,000 draws).

Table VIII lists the hypotheses and their verdicts in both phases.

TABLE VIII
HYPOTHESES, DECISION RULES AND VERDICTS. EACH RULE APPLIES TO THE MEDIAN SITE OF EVERY LINK–RADIO COMBINATION (12), OR OF EVERY LINK–RADIO–PARAMETER COMBINATION (48), AND PASSES ONLY IF IT HOLDS IN ALL. PHASE OF WRITING: P, BEFORE THE DEVELOPMENT RUNS; A, AFTER H4 FAILED, BEFORE THE RUNS CONCERNED. DEVELOPMENT: 2023 RERUN WITH D4 AND D5 FIXED; HELD-OUT: 2024, JUDGED BY THE FROZEN CODE. "EVENT" = EVENT-DRIVEN BATCHING; "BOUNDED" = BOUNDED EVENT-DRIVEN BATCHING.

		Claim	Rule	2023 (development)	2024 (held-out)
H1	P	Link up: Event raises alarms as fast as per-reading reporting	Event − ARQ+buffer alarm latency $\leq$ 1 slot	Pass, 12/12	Pass, 12/12
H2	P	Event costs about as much as batching, much less than per-reading ARQ	Event / batch $\leq$ 1.10 and ARQ+buffer / Event $\geq$ 1.3	Pass, 12/12	Pass, 12/12
H3	P	Onset in an outage: buffering delivers late, unconfirmed loses	descriptive	As expected	As expected
H4	P	Event's picture is as good as per-reading reporting's	Event $\geq$ ARQ+buffer − 5 pp	Fail, 38/48	Fail, 39/48
H5	A	Bounded's picture is as good as per-reading reporting's	Bounded $\geq$ ARQ+buffer − 1 pp	Pass, 48/48	Pass, 48/48
H6	A	Bounded costs no more than per-reading ARQ	Bounded $\leq$ ARQ+buffer energy	Pass, 12/12	Pass, 12/12
H7	A	The trigger does not delay alarms	Bounded − Event alarm latency $\leq$ 1 slot	Pass, 12/12	Pass, 12/12
H8	A	Newest-first packing makes the picture exact with the link up	100% within tolerance, no outage, every site	Fail as written; pass after D4	Pass
H9	A	H5 holds at tolerances L1 and L3	as H5	Pass, 48/48, 48/48	Pass, 48/48, 48/48
H10	A	H6 holds at tolerances L1 and L3	as H6	Pass, 12/12, 12/12	Pass, 12/12, 12/12

TABLE IX
HELD-OUT YEAR (2024): ALARM LATENCY WITH THE LINK UP (MEDIAN OVER SITES OF EACH SITE'S MEDIAN, HOURS) AND TRANSMISSION ENERGY (MEDIAN OVER SITES, J/DAY); LORAWAN SF7. ENERGIES OF THE EVENT-DRIVEN DESIGNS ARE AVERAGED OVER THE FOUR EVENT SETTINGS. ARQ: ARQ + BUFFER.

	Alarm latency (h)				Energy (J/day)			
Link	ARQ	Batch	Event	Bounded	ARQ	Batch	Event	Bounded
No outage	1.00	1.25	1.00	1.00	34.2	9.0	9.0	11.4
UVA	1.00	1.25	1.00	1.00	36.1	9.6	9.7	12.2
Weija	1.00	1.25	1.00	1.00	32.2	10.7	10.9	13.0
InWaterSense	1.00	1.50	1.00	1.00	37.1	18.4	18.7	21.4

C. Alarms with the link up (H1, H7)

With the link up at an event's onset, event-driven batching raises alarms at the server as fast as per-reading ARQ in both years (H1; Table IX). The median latency of about an hour is the detector's own: the CUSUM needs about an hour of evidence, and delivery adds nothing. Plain hourly batching is slower in every combination, by a median of 0.43–0.50 h in 2024. On the InWaterSense link the event-driven design is never slower than ARQ + buffer, and faster at SF12 in 2024 (on every radio in 2023), because an alarm probes at once while ARQ + buffer waits for its back-off schedule when a gap starts during the event. The bounded-error trigger does not delay alarms (H7). Counting all injected events, detected or not, the server knows of 34% within an hour with event-driven batching on the no-outage link at SF7, the same as with per-reading reporting, against 21% with hourly batching; within a day it knows of 58% with any of them. On the InWaterSense link the shares within an hour are 19% (bounded), 16% (ARQ + buffer), 12% (unconfirmed) and 11% (batch), and within a day 44%, 42%, 22% and 43%. The ceiling is the detector's 58.2% detection, not the reporting scheme.

D. Energy (H2, H6)

Alarm uploads are rare (a median of 0.13 alarms per node per day in 2024; mean 0.15), and most ride on readings that would have been sent within the hour anyway. Event-driven batching therefore costs 1.00×–1.02× as much as plain batching, and ARQ + buffer costs 1.41×–3.86× as much as event-driven batching (H2). The bounded-error trigger costs more, 1.05×–1.28× the energy of plain batching, because each trigger sends an extra upload between batches. Per-reading ARQ still uses 1.34×–2.55× as much energy as the bounded design (H6; every site-resampled 95% interval lies above 1.28×), against 1.32×–2.51× in the development year. At an hourly cadence an hourly batch holds a single reading, so batching cannot save energy there.

E. The real-time picture (H4, H5)

Table X and Fig. 4 show the server's picture. Plain event-driven batching fails H4 in both years: for DO, an estimate up to an hour old leaves the L2 tolerance between batches, and the picture is -9.0 to -6.9 percentage points less often within tolerance than with per-reading reporting (2023: -10.8 to -5.1). The failures occur with no outage at all, so they are a property of hourly batching, not of the gaps. The event detector does not help: these excursions are ordinary diel motion, not events. (In the first development run, H4 also failed for pH, 29/48 combinations passing; the pH failures were the rounding tie D5, not batching.)

The bounded-error trigger removes the failure. The bounded design's picture is within -0.01 to $+0.56$ percentage points of per-reading reporting's for every parameter, link and radio (H5; 2023: -0.02 to $+0.59$). With the link up and no radio loss it is within tolerance at every scored slot, by construction (H8 below).

TABLE X
HELD-OUT YEAR (2024): SHARE OF SCORED SLOTS WITHIN THE L2 TOLERANCE (MEDIAN OVER SITES; LORAWAN SF7; SERIES WITHOUT INJECTED EVENTS). ARQ: ARQ + BUFFER.

Link	Param.	Unconf.	ARQ	Batch	Event	Bounded
No outage	DO	100.0%	100.0%	92.0%	92.2%	100.0%
	pH	100.0%	100.0%	99.0%	99.1%	100.0%
	T	100.0%	100.0%	98.7%	98.7%	100.0%
	EC	100.0%	100.0%	96.7%	96.7%	100.0%
UVA	DO	98.8%	99.2%	91.3%	91.5%	99.2%
	pH	99.8%	99.8%	98.5%	98.6%	99.8%
	T	98.9%	98.8%	97.7%	97.7%	98.8%
	EC	99.2%	99.3%	96.5%	96.6%	99.3%
Weija	DO	90.6%	89.7%	83.8%	83.8%	90.0%
	pH	95.3%	95.0%	94.1%	94.1%	95.4%
	T	89.0%	88.2%	86.9%	87.0%	88.2%
	EC	90.5%	89.6%	87.2%	87.2%	89.7%
InWaterSense	DO	75.8%	72.4%	68.1%	68.1%	72.4%
	pH	90.9%	88.3%	90.1%	90.2%	90.6%
	T	74.1%	67.7%	67.2%	67.5%	68.2%
	EC	80.5%	74.6%	74.0%	73.8%	75.1%

TABLE XI
HELD-OUT YEAR (2024): DETECTED EVENTS WHOSE ONSET FELL IN A GAP (LORAWAN SF7, POOLED OVER SITES): NUMBER OF EVENTS, SHARE THE SERVER EVER LEARNS OF, AND MEDIAN LATENCY OF THOSE IT LEARNS OF (HOURS).

		Known ever			Median latency (h)			
Link	n	Unconf.	ARQ	Event	ARQ	Batch	Event	Bounded
InWaterSense	2,113	3.0%	98.8%	98.8%	27.5	26.6	24.3	24.0
Weija	661	4.1%	100.0%	100.0%	53.5	48.5	48.5	48.5
UVA	57	7.0%	100.0%	100.0%	28.5	24.9	24.9	24.9

Under the two water-quality gap records, unconfirmed reporting shows a *better* picture than ARQ + buffer for every parameter and radio (e.g., DO on InWaterSense at SF7: 75.8% vs 72.4%). When the link returns, an unconfirmed node delivers its next reading at once, while a buffering node waits for its next back-off probe. The back-off saves energy and costs timeliness right after gaps; unconfirmed reporting, of course, never delivers the readings lost in them.

F. Events that start during a gap (H3)

Table XI and Fig. 5 show the detected events whose onset fell in a gap. As expected (H3), every strategy has to wait for the link. Unconfirmed reporting learns of almost none of these events (at most 7.0% on any link and radio): the alarming readings fall inside the gap and are lost. The buffering strategies learn of at least 98.8%, but only after the gap; the gap, not the reporting scheme, sets this latency. Within it, the immediate alarm probe makes the event-driven designs faster than ARQ + buffer on the water-quality records.

G. Packing order and tolerance level (H8–H10)

On SF12 a frame holds only 3 readings, so an hourly batch at 15-min sampling needs two frames, and the duty cycle delays the second. With first-in-first-out packing the newest reading rode in the second frame, and in the first development run the server's picture on the no-outage link fell to 98.72% at worst instead of 100%. H8 predicted that packing the newest reading first would make it exact everywhere. As written, H8 failed: the picture fell short at one site (01467200, 5-min sampling, SF12), with a minimum of 99.998%. The cause was two implementation defects, each a departure from the stated design: the reading promoted to the first frame could be a placeholder for a sensor gap rather than the newest valid reading, and the node's copy of the server counted readings as known as soon as their upload began, although at 5-min sampling an SF12 upload spans several slots. Both were fixed (D4); the lesson is general: a node that mirrors its server must model when data arrive, not only whether. With the fix, newest-first packing is within tolerance at every scored slot of the no-outage link, on every site, parameter and radio, in both years, at 0.987–1.007 times the energy of first-in-first-out packing (2024).

TABLE XII
HELD-OUT YEAR (2024): ENERGY (MEDIAN J/DAY OVER SITES, LORAWAN SF7, SERIES WITHOUT INJECTED EVENTS) AT THREE TOLERANCE LEVELS, THE MEDIAN OVER SITES OF EACH SITE'S ARQ / BOUNDED ENERGY RATIO, AND THE LOWEST WITHIN-TOLERANCE SHARE OVER THE FOUR PARAMETERS (MEDIAN SITE). ARQ: ARQ + BUFFER.

		Energy (J/day)			Within tolerance	
Level	Link	ARQ	Bounded	ARQ/Bounded	Bounded	ARQ
L1 (tight)	No outage	34.2	19.4	1.56×	100.0%	100.0%
	InWaterSense	37.1	26.6	1.28×	61.7%	62.5%
L2	No outage	34.2	11.2	2.55×	100.0%	100.0%
	InWaterSense	37.1	21.1	1.65×	68.2%	67.7%
L3 (loose)	No outage	34.2	9.4	3.40×	100.0%	100.0%
	InWaterSense	37.1	19.1	1.84×	79.1%	78.3%

The tolerance level sets the price of the bound (Table XII). The picture stays within 1 percentage point of per-reading reporting at every level (H9), and per-reading ARQ uses 1.20×–1.57× the bounded design's energy at L1, 1.35×–2.58× at L2 and 1.40×–3.47× at L3 (H10).

H. Where the server's state comes from

On LoRaWAN, the evaluation above takes a confirmed uplink's acknowledgement, which comes from the network server, as proof that the application server shows the reading. If the node may not assume this, it keeps the bound for both possible server states until an application acknowledgement returns in the downlink of a later uplink (Sec. VII-A). On the development sites, this costs 3% to 18% more energy than trusting the network server, because the node uploads again whenever the older state could be out of tolerance; the picture of the median site stays within -0.02 percentage points of per-reading reporting, and per-reading ARQ still uses 1.29×–2.00× as much energy. On NB-IoT the question does not arise: its confirmation is the application's reply.

The answer to RQ4 is yes, with the link up, and on data not used to design the scheme. Bounded event-driven batching delivers every reading the node generates, raises alarms as fast as per-reading reporting, keeps the server's picture within a chosen tolerance as often as per-reading reporting does, and costs 1.05×–1.28× the energy of plain hourly batching, while per-reading ARQ uses 1.34×–2.55× as much energy on the

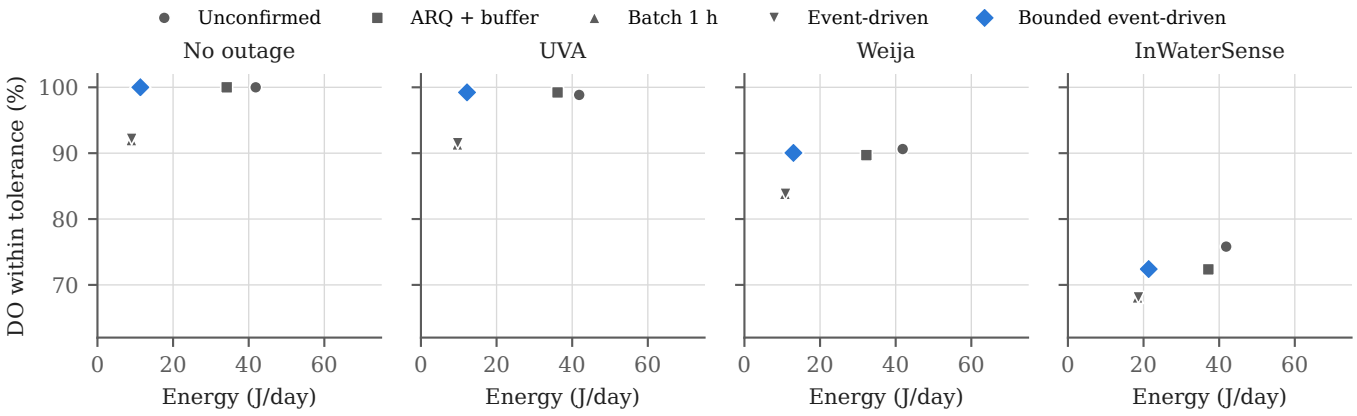


Fig. 4. Held-out year (2024): energy against the real-time picture of DO (share of slots within the L2 tolerance), LoRaWAN SF7, median over sites, one panel per link, all panels on the same vertical scale. Batch 1 h and event-driven batching nearly coincide. The bounded design reaches the picture of per-reading reporting at a fraction of its energy.

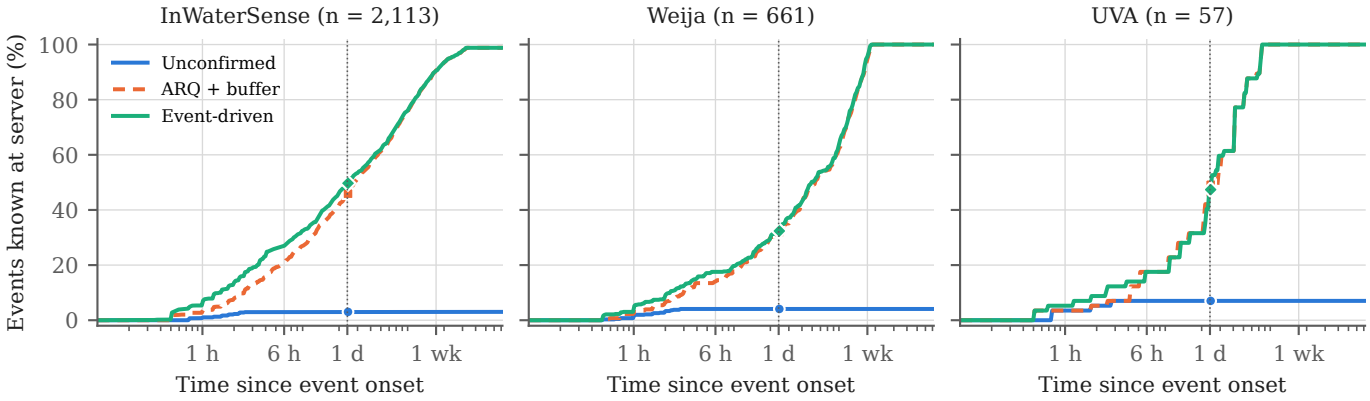


Fig. 5. Held-out year (2024): detected events whose onset fell in a gap, share known at the server by the time on the horizontal axis (LoRaWAN SF7, pooled over sites; markers at 1 day). Unconfirmed reporting learns of almost none; the buffering strategies learn of nearly all, after the gap.

held-out year. What no scheme can fix is the gap itself: events and data during a gap reach the server only when the link returns.

VIII. Discussion

A. Practical guidance

The results suggest six rules for battery-powered water-quality telemetry on LPWANs.

(1) *Know which regime loses your data.* Short radio losses and long delivery gaps call for different remedies, and in the two water-quality deployments studied the long gaps held most of the missing data. A delivery log with frame counters, a note of when the gateway and server were down, and the node's own status are enough to tell the regimes and their causes apart (Secs. III-B and IV). Simulations and estimators that assume geometric gap lengths should be checked against such a log.

(2) *Do not fight long gaps with retransmissions.* ARQ is useful against radio losses only; under any of the timing models it recovers under a fifth of the readings missing in the long-gap record. Confirm, buffer and back off instead; on NB-IoT, back off even if nothing is buffered, because searching for a cell in every slot of a gap cost up to 6.53× the energy of backing off in the swept range (Sec. V).

(3) *Give the buffer room and the backlog manners.* Tens of kilobytes held the longest backlog in these records, and writing it to flash costs almost nothing. At high spreading factors the duty cycle, not the buffer, limits recovery: interleave live readings ahead of the backlog. Where many nodes share a gateway, randomise the recovery probes, and at SF12 acknowledge backlogs in chunks rather than frame by frame (Sec. V-E).

(4) *Batch routine data, and make the batch leave early when it matters.* Hourly batching delivers every reading for a fraction of per-reading energy at SF7 and on NB-IoT; at SF12 its advantage depends on the energy of a confirmed frame and should be measured. Two triggers close its two gaps: an event alarm, for timeliness, and a bounded-error trigger that mirrors the server's estimate, for the real-time picture. The tolerance is then the operator's energy knob, and a tolerance at the USGS "excellent" accuracy of the sensors costs well under the energy of per-reading ARQ (Sec. VII). The node must model when its data arrive, and, on LoRaWAN, whether the application has

them.

(5) *Use the site's own diel pattern as the interim estimate, and judge reconstruction against it.* Between deliveries, a site's own day–night pattern is the best simple estimator for DO and pH; neighbours help temperature, and help most when bridging a gap after the data return. A reconstruction method evaluated only against the last value will overstate what spatial information adds (Sec. VI). Contemporary deep imputers, trained per river, reduce the error of DO and pH when bridging a day-long gap, but keep fewer short gaps within tolerance than the simple estimators and no more day-long ones (Sec. efsec:rec-deep).

(6) *Engineer the gaps themselves.* No reporting scheme shortens a gap; events that start during one reach the server only when it ends. Gateway diversity masked a month-long gateway outage at UVA without losing a frame (Sec. IV-C), and deployment reports show that watchdog resets bound how long silent gateway failures last [6], [7]. How long system outages last is partly an operations variable.

B. Validity and limitations

Delivery records. The loss evidence comes from two water-quality deployments and two LoRaWAN networks; it supports statements about these records, not a law for all deployments. The Weija record has few long gaps and serves as qualitative support only, and its unflagged interpolation hides some short gaps. The cause of a gap could be checked only partly (Sec. III-B); buffering's recovery is therefore an upper bound, tested with up to half of the long-gap slots reassigned to a node that was off. UVA's "system" outages include the operators' own logging pipeline, which from a user's point of view is still data that did not arrive.

Energy without hardware. No hardware was built for this study. The energy model rests on published measurements of specific modules and network settings, with an ideal battery, and an independent measurement on a different board and stack differs from it severalfold in absolute terms (Sec. III-D). The results are therefore ratios between strategies on the same radio, tested against a factor-of-two uncertainty on every measured component (Sec. V-F); lifetimes are illustrative. The energy an NB-IoT modem spends searching when there is no network has no published measurement that could be found, so it was swept. A measurement campaign on a LoRaWAN and an NB-IoT node, including a deliberate gateway disconnection, is the most important next step.

What the per-node model leaves out. Clock faults, reordering and storage failure [7], [32] are not simulated. Many nodes recovering at once are studied with a separate, simplified model of one gateway (Sec. V-E); multi-gateway reception, adaptive data rate and real channel conditions would change its numbers, but not the two mechanisms it shows, synchronised probes and acknowledgement capacity.

Events and detection. The events are synthetic steps and ramps injected into real series, of realistic size and physically plausible, and the detector, whose parameters were fixed in advance, catches slow ramps poorly (13% of 5σ ramps). The latency comparisons concern how fast the server learns of a detected event; the all-event shares show how much of an operator's view is set by detection rather than by reporting.

Rivers and tolerances. The river data cover two years in one country, and some lower-Delaware sites are tidal, where conductivity follows the tide rather than the day. The held-out year shares most sites with the development year (29 of 33), so it tests the design on new data, not on new rivers. The tolerances are at or inside the USGS accuracy ratings, not regulatory limits; the sensitivity analysis shows how the results change with them. The server's estimator is deliberately simple, and a learned estimator could shorten the bounded design's extra uploads, but not the gap a long outage leaves.

Pre-specification. The development hypotheses and decision rules were written down before their runs and kept with the code; the confirmatory plan and code were frozen and hashed before the held-out data were downloaded. Neither was registered with a third party. Every verdict is reported, including the failures and the defect found during the revision (D5), with its effect.

C. Conclusion

In the four records studied, data are lost in two regimes: short radio losses, and rarer long delivery gaps that, in the two water-quality deployments, hold most of the missing data. Remedies designed for short, independent losses fit the first regime and not the second. Retransmission spends energy on gaps it cannot bridge, and the reconstruction estimators evaluated, from a site's own history to deep imputers, keep most day-long gaps outside the tolerance. Buffering with back-off delivers every reading the node generates at an energy close to that of not recovering at all, and hourly batching makes it cheaper still. A reporting design that batches hourly, sends alarms at once, and leaves early whenever the server's estimate would leave tolerance matched per-reading reporting on alarm latency and on the real-time picture on a held-out year of river data, while per-reading ARQ used $1.34\times$–$2.55\times$ as much energy. The remaining gap is the outage itself, which only network engineering, such as gateway diversity and watchdogs, can shorten. Future work includes hardware measurements of the design, longer and tropical outage records, the energy of NB-IoT cell search without a network, and recovery of many nodes across several gateways.

Data and Code Availability

The USGS [52], InWaterSense [41], LoED [43] and UVA [42] data are public. The Weija records were provided by the deployment's operators [2]; their raw values are not redistributed, but the delivery trace derived from them (the time of every record of every stream, and whether it counts as a genuine delivery) is. The code, configurations, seeds, environment lock files, data-acquisition scripts, input checksums, the frozen confirmatory plan with the hashes of the frozen code, and the derived traces are archived at [archive DOI].

Acknowledgment

This work received no external funding.

REFERENCES

[1] K. S. Adu-Manu, C. Tapparello, W. Heinzelman, F. A. Katsriku, and J.-D. Abdulai, "Water quality monitoring using wireless sensor networks: Current trends and future research directions," *ACM Transactions on Sensor Networks*, vol. 13, no. 1, pp. 1–41, 2017.

[2] K. S. Adu-Manu, F. A. Katsriku, J.-D. Abdulai, and F. Engmann, "Smart river monitoring using wireless sensor networks," *Wireless Communications and Mobile Computing*, vol. 2020, pp. 1–19, 2020, art. no. 8897126.

[3] F. Ahmedi and L. Ahmedi, "Dataset on water quality monitoring from a wireless sensor network in a river in Kosovo," *Data in Brief*, vol. 44, p. 108486, 2022.

[4] T. Le Dinh, W. Hu, P. Sikka, P. Corke, L. Overs, and S. Brosnan, "Design and deployment of a remote robust sensor network: Experiences from an outdoor water quality monitoring network," in *Proc. 32nd IEEE Conf. Local Computer Networks (LCN)*. IEEE, 2007, pp. 799–806.

[5] K. Langendoen, A. Baggio, and O. Visser, "Murphy loves potatoes: Experiences from a pilot sensor network deployment in precision agriculture," in *Proc. 20th IEEE Int. Parallel and Distributed Processing Symp. (IPDPS)*. IEEE, 2006, p. 8 pp.

[6] F. Ingelrest, G. Barrenetxea, G. Schaefer, M. Vetterli, O. Couach, and M. Parlange, "SensorScope: Application-specific sensor network for environmental monitoring," *ACM Transactions on Sensor Networks*, vol. 6, no. 2, pp. 17:1–17:32, 2010.

[7] M. Brzozowski, M. Frohberg, and P. Langendoerfer, "Real-world deployments of sensor networks: Practical lessons for researchers," in *Wired/Wireless Internet Communications (WWIC 2018), LNCS vol. 11618*. Springer International Publishing, 2018, pp. 42–53.

[8] P. P. Puluckul and M. Weyn, "Experiences with sub-arctic sensor network deployment," in *Proc. IEEE Wireless Communications and Networking Conf. (WCNC)*. IEEE, 2025, pp. 1–6.

[9] M. Zennaro, E. Pietrosemoli, A. Soltobaev, T. Sultanov, and B. Moldobekov, "From mountains to data: low-cost weather stations in Kyrgyzstan's challenging terrain," *Frontiers in Communications and Networks*, vol. 6, 2025, art. no. 1505375.

[10] B. Sinopoli, L. Schenato, M. Franceschetti, K. Poolla, M. Jordan, and S. Sastry, "Kalman filtering with intermittent observations," *IEEE Transactions on Automatic Control*, vol. 49, no. 9, pp. 1453–1464, 2004.

[11] K. Huang, W. Liu, Y. Li, and B. Vucetic, "To retransmit or not: Real-time remote estimation in wireless networked control," in *Proc. IEEE Int. Conf. Communications (ICC)*. IEEE, 2019, pp. 1–7.

[12] K. Huang, W. Liu, M. Shirvanimoghaddam, Y. Li, and B. Vucetic, "Real-time remote estimation with hybrid arq in wireless networked control," *IEEE Transactions on Wireless Communications*, vol. 19, no. 5, pp. 3490–3504, 2020.

[13] L. Kong, M. Xia, X.-Y. Liu, G. Chen, Y. Gu, M.-Y. Wu, and X. Liu, "Data loss and reconstruction in wireless sensor networks," *IEEE Transactions on Parallel and Distributed Systems*, vol. 25, no. 11, pp. 2818–2828, 2014.

[14] A. Cini, I. Marisca, and C. Alippi, "Filling the G_ap_s: Multivariate time series imputation by graph neural networks," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022, arXiv:2108.00298.

[15] I. Marisca, A. Cini, and C. Alippi, "Learning to reconstruct missing data from spatiotemporal graphs with sparse observations," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.

[16] Q. Chang, D. Tao, J. Wang, and R. Gao, "Deep compressed sensing based data imputation for urban environmental monitoring," *ACM Transactions on Sensor Networks*, vol. 20, no. 1, pp. 1–21, 2023, art. no. 17.

[17] J. Zhao and R. Govindan, "Understanding packet delivery performance in dense wireless sensor networks," in *Proc. 1st Int. Conf. Embedded Networked Sensor Systems (SenSys)*. ACM, 2003, pp. 1–13.

[18] L. Bhatia, M. Breza, R. Marfievici, and J. A. McCann, "LoED: The LoRaWAN at the edge dataset," in *Proc. 3rd Workshop on Data: Acquisition To Analysis (DATA '20)*. ACM, 2020, pp. 7–8.

[19] F. Nikseresht, V. A. L. Sobral, J. L. Goodall, and B. Campbell, "Dataset: Long-term LoRaWAN communication metadata from an urban deployment," in *Proc. 12th ACM Int. Conf. Systems for Energy-Efficient Buildings, Cities, and Transportation (BuildSys '25)*. ACM, 2025, pp. 412–417.

[20] R. D. Yates, Y. Sun, D. R. Brown, S. K. Kaul, E. Modiano, and S. Ulukus, "Age of information: An introduction and survey," *IEEE Journal on Selected Areas in Communications*, vol. 39, no. 5, pp. 1183–1210, 2021.

[21] B. Sun and N. Pappas, "Resource-constrained semantic-aware remote estimation with overlapping sensor coverage," in *Proc. ACM MobiHoc*, 2026, to appear; arXiv:2609.07563.

[22] P. J. Marcelis, V. Rao, and R. V. Prasad, "Dare: Data recovery through application layer coding for lorawan," in *Proceedings of the Second International Conference on Internet-of-Things Design and Implementation*. ACM, 2017, pp. 97–108.

[23] C. Luo, F. Wu, J. Sun, and C. W. Chen, "Compressive data gathering for large-scale wireless sensor networks," in *Proc. 15th Annu. Int. Conf. Mobile Computing and Networking (MobiCom)*. ACM, 2009, pp. 145–156.

[24] J. Cheng, Q. Ye, H. Jiang, D. Wang, and C. Wang, "STCDG: An efficient data gathering algorithm based on matrix completion for wireless sensor networks," *IEEE Transactions on Wireless Communications*, vol. 12, no. 2, pp. 850–861, 2013.

[25] X. Piao, Y. Hu, Y. Sun, B. Yin, and J. Gao, "Correlated spatio-temporal data collection in wireless sensor networks based on low rank matrix approximation and optimized node sampling," *Sensors*, vol. 14, no. 12, pp. 23 137–23 158, 2014.

[26] D. Cao, Y. Wang, J. Duan, C. Zhang, X. Zhu, C. Huang, Y. Tong, B. Xu, J. Bai, J. Tong, and Q. Zhang, "Spectral temporal graph neural network for multivariate time-series forecasting," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 17 766–17 778.

[27] W. Du, D. Côté, and Y. Liu, "Saits: Self-attention-based imputation for time series," *Expert Systems with Applications*, vol. 219, p. 119619, 2023.

[28] T. Nie, G. Qin, W. Ma, Y. Mei, and J. Sun, "Imputeformer: Low rankness-induced transformers for generalizable spatiotemporal imputation," in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. ACM, 2024, pp. 2260–2271.

[29] B. Jing, D. Zhou, K. Ren, and C. Yang, "Causality-aware spatiotemporal graph neural networks for spatiotemporal time series imputation," in *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. ACM, 2024, pp. 1027–1037.

[30] E. E. Peterson and J. M. Ver Hoef, "A mixed-model moving-average approach to geostatistical modeling in stream networks," *Ecology*, vol. 91, no. 3, pp. 644–651, 2010.

[31] P. McDonald, D. Geraghty, I. Humphreys, S. Farrell, and V. Cahill, "Sensor network with delay tolerance (SeNDT)," in *2007 16th International Conference on Computer Communications and Networks*. IEEE, 2007, pp. 1333–1338.

[32] M. J. Murphy, Ø. Tveito, E. F. Kleiven, I. Rais, E. M. Soininen, J. M. Bjørndalen, and O. Anshus, "Experiences building and deploying wireless sensor nodes for the arctic tundra," in *Proc. IEEE/ACM 21st Int. Symp. Cluster, Cloud and Internet Computing (CCGrid)*. IEEE, 2021, pp. 376–385.

[33] L. Casals, B. Mir, R. Vidal, and C. Gomez, "Modeling the energy performance of LoRaWAN," *Sensors*, vol. 17, no. 10, p. 2364, 2017.

[34] F. Michelinakis, A. S. Al-Selwi, M. Capuzzo, A. Zanella, K. Mahmood, and A. Elmokashfi, "Dissecting energy consumption of NB-IoT devices empirically," *IEEE Internet of Things Journal*, vol. 8, no. 2, pp. 1224–1242, 2021.

[35] A. Sørensen, H. Wang, M. J. Remy, N. Kjettrup, R. B. Sørensen, J. J. Nielsen, P. Popovski, and G. C. Madueño, "Modeling and experimental validation for battery lifetime estimation in NB-IoT and LTE-M," *IEEE Internet of Things Journal*, vol. 9, no. 12, pp. 9804–9819, 2022.

[36] A. Azari, C. Stefanovic, P. Popovski, and C. Cavdar, "On the latency-energy performance of NB-IoT systems in providing wide-area IoT connectivity," *IEEE Transactions on Green Communications and Networking*, vol. 4, no. 1, pp. 57–68, 2020.

[37] S. Ugwuanyi, G. Paul, and J. Irvine, "Survey of IoT for developing countries: Performance analysis of LoRaWAN and cellular NB-IoT networks," *Electronics*, vol. 10, no. 18, p. 2224, 2021, art. no. 2224.

[38] G. M. Dias, B. Bellalta, and S. Oechsner, "A survey about prediction-based data reduction in wireless sensor networks," *ACM Computing Surveys*, vol. 49, no. 3, pp. 1–35, 2016.

[39] S. Santini and K. Römer, "An adaptive strategy for quality-based data reduction in wireless sensor networks," in *Proc. 3rd Int. Conf. Networked Sensing Systems (INSS)*, 2006.

[40] D. Arnaiz, F. Moll, E. Alarcón, and X. Vilajosana, "Energy and relevance-aware adaptive monitoring method for wireless sensor nodes with hard energy constraints," *Integration*, vol. 94, p. 102097, 2024.

[41] L. Ahmedi, "InWaterSense dataset: Data from a wireless sensor network on water quality monitoring in a river in Kosovo," Mendeley Data, V1, 2021.

[42] F. Nikseresht, V. A. L. Sobral, J. L. Goodall, and B. Campbell, "Long-term LoRaWAN communication metadata from an urban deployment," University of Virginia Dataverse (LibraData), 2025.

[43] L. Bhatia, M. Breza, R. Marfievici, and J. A. McCann, "LoED: The LoRaWAN at the edge dataset," Zenodo, 2020.
[44] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.
[45] H. Akaike, "A new look at the statistical model identification," *IEEE Transactions on Automatic Control*, vol. 19, no. 6, pp. 716–723, 1974.
[46] *Waspmote Plug & Sense! Technical Guide (product line v15)*, Libelium Comunicaciones Distribuidas, 2020, document version v9.2.
[47] LoRa Alliance Technical Committee Regional Parameters Workgroup, "RP002-1.0.3 LoRaWAN regional parameters," LoRa Alliance, Tech. Rep., 2021.
[48] LoRa Alliance Technical Committee, "LoRaWAN specification v1.0.2," LoRa Alliance, Tech. Rep., 2016, sec. 18.4, Data-rate adaptation during message retransmissions.
[49] ——, "LoRaWAN L2 1.0.4 specification (TS001-1.0.4)," LoRa Alliance, Tech. Rep., 2020, sec. 4.3.1.3, Retransmission procedure; Sec. 5.3, LinkADRReq (NbTrans).
[50] *W25Q128JV 3V 128M-bit Serial Flash Memory, Datasheet, Rev. F*, Winbond Electronics, 2018, sec. 9.4 (DC characteristics) and AC electrical characteristics.
[51] R. J. Wagner, R. W. Boulger, C. J. Oblinger, and B. A. Smith, *Guidelines and standard procedures for continuous water-quality monitors: Station operation, record computation, and data reporting*, 2006.
[52] U.S. Geological Survey, "USGS water data for the nation (National Water Information System)," 2024, instantaneous values for 2023, accessed Oct. 2026.
[53] B. B. Benson and D. Krause, "The concentration and isotopic fractionation of oxygen dissolved in freshwater and seawater in equilibrium with the atmosphere," *Limnology and Oceanography*, vol. 29, no. 3, pp. 620–632, 1984.
[54] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, p. 80, 1945.
[55] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1–2, pp. 100–115, 1954.